

StOMR: Stateful Oblivious Message Retrieval

Charles Gouert
NVIDIA
cgouert@nvidia.com

Keewoo Lee
Ethereum Foundation
keewoo.lee@ethereum.org

Dimitris Mouris*
Snap Inc.
jimouris@udel.edu

Yiannis Tselekounis
Royal Holloway University of London
yiannis.tselekounis@rhul.ac.uk

Jean-Luc Watson
NVIDIA
jwatson@nvidia.com

Abstract

Oblivious Message Retrieval (OMR) enables recipients to retrieve messages from an untrusted server without revealing which messages correspond to them. OMR is useful for privacy-preserving systems such as anonymous messaging and blockchains. Existing constructions are inherently built in the public-key setting and use Fully Homomorphic Encryption (FHE) to allow a detector to scan a public bulletin board and produce a compact encrypted digest containing only relevant messages. However, this design incurs a major practical limitation: sender-published signals are large (hundreds of kilobytes), leading to significant bandwidth, storage, and on-chain costs.

In this work, we explore the efficiency gains achievable by moving to a private, stateful setting, and introduce *stateful OMR*, a new model in which senders and receivers maintain lightweight private state to enable more efficient anonymous communication. Based on this model, we design StOMR, a protocol that significantly reduces the size of sender signals compared to prior approaches. To support the required shared state, we further introduce an *OMR Key Encapsulation Mechanism (OMR-KEM)*, which allows users to privately establish shared state and instantiate stateful OMR without out-of-band channels.

We implement StOMR in C++ and evaluate it against the state-of-the-art systems PerfOMR (USENIX Security '24), SophOMR (USENIX Security '26), and InstantOMR (USENIX Security '26). Our results show that StOMR preserves detector performance while achieving a signal size approximately 550×, 650×, and 178× smaller than PerfOMR, SophOMR, and InstantOMR, respectively.

CCS Concepts

• Security and privacy → Cryptography.

Keywords

Oblivious message retrieval, fully homomorphic encryption, receiver privacy

*Work completed while at Nillion.

Authors listed alphabetically;
see <https://ams.org/profession/leaders/CultureStatement04.pdf>.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '26, The Hague, Netherlands
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
<https://doi.org/10.1145/3830454.3846701>

ACM Reference Format:

Charles Gouert, Keewoo Lee, Dimitris Mouris, Yiannis Tselekounis, and Jean-Luc Watson. 2026. StOMR: Stateful Oblivious Message Retrieval. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3846701>

1 Introduction

Oblivious Message Retrieval (OMR), introduced by Liu and Tromer [45], is a key primitive for reconciling efficiency with strong metadata privacy in settings where large volumes of encrypted data are publicly available. In such systems, recipients must identify which ciphertexts correspond to them without revealing this interest to external observers. Naïvely scanning all data locally is computationally prohibitive, while relying on a third-party server introduces the risk of leaking access patterns and thus sensitive metadata.

OMR addresses this challenge by enabling users to delegate the task of message detection to an untrusted server – often referred to as a detector – without disclosing which messages are relevant to them. Concretely, the detector processes the entire dataset and returns a compact, encrypted response that only the intended recipient can interpret, ensuring that no information about the user's queries or matches is exposed.

A key motivating application of OMR is privacy-preserving cryptocurrencies [10, 13, 53, 59], also known as *privacy coins*, designed to enhance anonymity and untraceability by masking sender, receiver, and transaction amounts [11], unlike public ledgers such as Bitcoin [52] and Ethereum [17]. The most prominent networks include ZCash [13], Monero [53, 59], and Aztec Network [10], among others, while even public networks like Ethereum can support private transactions with stealth addresses [61]. However, encrypted transactions on a public blockchain require recipients to identify their own outputs. While the underlying cryptographic protocols hide transaction contents and addresses, recipients must still identify relevant messages among the growing set of published ciphertexts. One solution is for each user to download and locally scan all messages, but this approach quickly becomes impractical. OMR enables users to identify relevant ciphertexts efficiently, while ensuring that the server learns nothing about which ciphertexts match the recipient. Beyond blockchains, OMR has many applications in anonymous messaging systems [24, 56, 58], where metadata-privacy is important, and privacy-preserving analytics [15, 23, 51].

Existing OMR systems [41, 42, 47] are built in the public-key setting, where senders publish publicly verifiable encrypted signals that any receiver can later query. The three most recent OMR works, PerfOMR [47], SophOMR [41], and InstantOMR [42], rely

on fully homomorphic encryption (FHE) [16, 28, 30, 32, 60] to allow the detector to scan a public bulletin board and return a compact encrypted digest containing only the relevant messages. This design mirrors the flexibility of public-key cryptography, but also inherits its inefficiencies. This raises a natural question:

Can we obtain substantial efficiency gains by moving to a private, stateful setting – analogous to the improvements achieved when replacing public-key primitives with symmetric ones?

1.1 Our Contribution

We answer this question in the affirmative and present the following results:

- We present the notion of *stateful OMR*, in which the sender and the receiver maintain private states that enable private and anonymous communication.
- We present StOMR, a stateful OMR protocol, and prove its security assuming IND-CPA security of FHE and pseudorandom functions (PRFs). By leveraging shared state, StOMR significantly reduces the size of sender-published signals from kilobytes in prior constructions to just a few bytes, substantially reducing bandwidth requirements and on-chain publication costs.
- As instantiating stateful OMR requires a shared state between sender and receiver, we introduce the notion of OMR Key Encapsulation Mechanism (*OMR-KEM*), which enables users to privately establish shared states and initialize stateful OMR without requiring an out-of-band channel. We show how to generically construct OMR-KEM assuming OMR and any public key encryption scheme, satisfying a notion of receiver privacy.
- We implement StOMR in C++ on two platforms: a CPU version (StOMR-CPU) on top of OpenFHE and a GPU-accelerated implementation (StOMR-GPU) backed by custom BFV CUDA kernels, and compare both with SophOMR and InstantOMR across multiple factors, including signal size and detector/sender/receiver cost. StOMR signal sizes are 550×, 650×, and 178× smaller than PerfOMR, SophOMR, and InstantOMR, respectively. Publishing a StOMR digest is also cheaper; 2.7×, 3×, and 1.5× on Ethereum, and 564×, 667×, and 179× on Zcash. StOMR-CPU achieves up to 3.6× lower latency than SophOMR and over 15× lower than InstantOMR at 2^{16} transactions; StOMR-GPU compounds this with an additional 1.6–3.7× latency reduction over StOMR-CPU, reaching 3.3–13.6× faster than SophOMR and over 25× faster than InstantOMR at the same scale.

1.2 The Model

We follow the model of prior works [41, 42, 45, 47]. In particular, we consider the following parties:

- **Bulletin Board:** A public append-only ledger where senders post messages. The bulletin board stores the posted entries and makes them available for processing.
- **Sender:** A party that wishes to deliver a payload x (i.e., an arbitrary bitstring) to a receiver by posting it to the bulletin board.
- **Receiver:** A party that retrieves messages intended for it with the help of a *Detector*. The receiver processes information derived from the bulletin board to recover the messages addressed to it.

- **Detector:** A service that scans the bulletin board and processes its contents to help receivers efficiently and privately fetch their messages.

Threat Model. Our threat model follows the standard setting used in prior OMR work [41, 42, 45, 47]. We consider a passive adversary whose goal is to link messages to their intended receivers while otherwise adhering to the protocol. The adversary observes the public bulletin board, all public keys, and all communications exchanged among the system participants. It may also collude with any party in the system, except for the sender and receiver of the target messages.

1.3 Technical Overview

Stateful OMR definition. Stateful Oblivious Message Retrieval (S-OMR) extends the standard OMR functionality by introducing evolving sender and receiver states that capture interaction history across multiple signal and decoding operations. Syntactically, S-OMR replaces the static key generation of OMR with a state generation algorithm *StateGen* that outputs a sender state st_{snd} , a receiver state st_{rcv} , and a detection key pk_{det} . The signaling algorithm becomes stateful, taking st_{snd} as input and producing both a signal sig and an updated state st'_{snd} , thereby allowing repeated message publication without reinitializing the system. On the receiver side, decoding is also stateful: given a digest $dgst$, the current receiver state st_{rcv} , the algorithm outputs a list of recovered payloads together with an updated state st'_{rcv} , enabling progressive retrieval over multiple digest computations while maintaining consistency with previously processed bulletin boards. A way in which this syntax diverges from standard OMR is that the detection key pk_{det} is updated as part of each decoding operation, as a direct consequence of the stateful nature of Decode.

From a security perspective, S-OMR strengthens the standard receiver privacy notion to a multi-query, stateful setting in which adversaries interact with oracles that simulate evolving sender and receiver states. In the receiver privacy experiment, the adversary is given access to a stateful signaling oracle and a stateful decoding oracle that operate consistently with respect to a hidden bit $b \in \{0, 1\}$, while only being exposed to the corresponding detection keys and signals or digests derived from one of the two worlds. Security requires that no PPT adversary can distinguish which receiver state is being used, even after adaptively observing multiple signal generations and digest/decoding interactions. This captures privacy in the presence of state evolution, ensuring that repeated use of the system does not leak information about the underlying receiver identity or targeted message structure beyond what is implied by correctness.

FHE-based Stateful OMR construction. We construct StOMR from PRFs and FHE in the BFV setting. The public parameters consist of the BFV plaintext modulus p , ciphertext modulus q , and slot dimension N . At state generation, the sender samples a PRF key $k \leftarrow \mathcal{K}$ and initializes a counter-based sender state $st_{snd} = (1, k)$, while the receiver is initialized with an FHE key pair (pk_{fhe}, sk_{fhe}) and state $st_{rcv} = (pk_{fhe}, sk_{fhe}, 1, k)$. The detection key pk_{det} consists of an FHE public key together with an FHE-encrypted batch of PRF outputs corresponding to the first P sender indices.

The signaling algorithm is stateful and deterministic given (i, k) : it outputs $\text{sig} = \text{PRF}_k(i)$ and increments the internal counter. Digesting is performed over the bulletin board by first batching signals and then using FHE evaluation to identify matching PRF outputs. In particular, equality with the expected PRF value is computed homomorphically and combined across slots via rotation-and-multiplication to detect full matches, producing a compressed indicator vector that selects relevant payloads without revealing their positions. The decoding algorithm uses the FHE secret key to decrypt the compressed output, recovers a prefix of valid messages, advances the receiver index, and refreshes pk_{det} by homomorphically generating and encrypting the next window of PRF values.

We achieve efficiency via careful SIMD batching in the underlying BFV scheme. Each PRF output of m bits is encoded into $s = \lceil \frac{m}{\log_2 P} \rceil$ slots, where each ciphertext provides N SIMD slots. The receiver's query ciphertext replicates each target identifier across slots, while the sender packs multiple identifiers sequentially, yielding an asymmetric layout that enables parallel comparisons against many candidates. Concretely, identifiers are grouped into batches of size $b = \frac{N}{P \cdot s}$, and each group is encoded into a single ciphertext using BFV batching. Here P denotes the number or pertinent messages.

Equality testing is performed homomorphically across slots, after which a rotation-and-multiplication reduction (an AND-tree over s slots) collapses each identifier block into a single indicator bit. This produces a compact pertinence vector marking matches, which is then fed into the final compression step (following SophOMR [41]). This SIMD structure enables a single homomorphic evaluation to simultaneously process many identifiers, thereby amortizing the cost of expensive FHE operations over the full batch.

OMR-KEM definition. OMR-KEM enables senders to communicate private keys to receivers so that they later switch to a communication based on stateful OMR. In OMR-KEM, the payload space is replaced by a key space and message transmission is abstracted into key encapsulation. Syntactically, the bulletin board abstraction and the Digest/Decode pipeline are preserved, but the Signal operation is replaced by Encap, and payloads tagged with signals (as one in the original OMR) are replaced by ciphertexts produced under a public encapsulation key. Correspondingly, Decap, recovers a list of session keys rather than arbitrary payloads. This shift removes the need for an explicit signaling key, folding receiver targeting into the encapsulation mechanism itself, while retaining the detection key to support selective retrieval via Digest. In particular, in OMR-KEM, there is no separate Signal algorithm or Signal key. Instead, receiver targeting is achieved implicitly through Encap executed over the receiver's public key. The ciphertext now plays a dual role – it both encodes the key and serves as the object that will be tested during Digest using the detection key.

From a security perspective, OMR-KEM ensures receiver privacy (similar to OMR) under a natural substitution: indistinguishability of signals is replaced by indistinguishability of encapsulated ciphertexts. In addition, OMR-KEM introduces a standard IND-CPA requirement on the encapsulated keys, ensuring that even if an adversary observes the bulletin board and the challenge ciphertext, it cannot distinguish the real encapsulated key from random. Thus, while OMR focuses on hiding which messages are addressed

to a receiver, OMR-KEM simultaneously guarantees both receiver anonymity and semantic security of the derived keys, aligning the primitive with conventional KEM security notions while preserving the oblivious retrieval structure.

OMR-KEM construction. We construct OMR-KEM from OMR and public-key encryption (PKE) with receiver privacy. The key generation algorithm runs the OMR key generation to obtain $(\text{sk}_{\text{omr}}, \text{pk}_{\text{sig}}, \text{pk}_{\text{det}})$, and independently runs the PKE key generation to obtain $(\text{pk}_{\text{pke}}, \text{sk}_{\text{pke}})$. The OMR secret key is augmented with the PKE secret key, the public encapsulation key consists of the OMR signal key together with the PKE encryption key, and the detection key is inherited unchanged from the underlying OMR scheme. To encapsulate, the sender first computes an OMR signal $\text{sig} \leftarrow \text{Signal}(\text{pk}_{\text{sig}})$, samples a random session key $k \leftarrow \mathcal{K}$, and encrypts it under the PKE public key to obtain $c \leftarrow \text{Enc}(\text{pk}_{\text{pke}}, k)$; the output is the pair $(k, (c, \text{sig}))$. Digesting is performed exactly as in the underlying OMR scheme by treating each bulletin board entry as a tagged ciphertext pair. Decapsulation first applies the OMR decoding algorithm to recover the list of relevant ciphertexts and then decrypts each ciphertext using the PKE secret key to recover the corresponding session keys. Correctness (resp. receiver privacy) of OMR-KEM reduces the correctness (resp. receiver privacy) property of the underlying OMR and PKE schemes.

Implementation & benchmarking. We implement StOMR in two versions – StOMR-CPU uses the OpenFHE BFV backend, as do each of our baselines (SophOMR [41] and InstantOMR [42]). All CPU schemes are evaluated on a 64-core AMD EPYC 7702P server with 256 GB RAM. All schemes exploit multi-threading via OpenFHE's native thread pool (64 threads), and are evaluated under identical ciphertext packing and batching strategies to ensure fairness. StOMR-GPU uses a Microsoft SEAL-based interface and a custom set of BFV CUDA kernels that natively support batched queries, allowing efficient parallel query processing on the GPU. The GPU-based results are reported for an NVIDIA RTX 6000 Ada graphics card with 48GB memory.

We parameterize BFV for StOMR with a polynomial degree of 2^{16} and a 20-bit plaintext modulus, achieving at least 128-bit security with multiplicative depth 22. SophOMR uses the same polynomial degree with an expanded ciphertext modulus (six additional primes) to support range checks under comparable security guarantees. Across all protocols, PRF outputs are fixed to 32 bits, with each output occupying two ciphertext slots. Following prior work [41, 42], we fix the payload size to 612 bytes, corresponding to standard Zcash transaction sizes [36].

StOMR consistently outperforms SophOMR, achieving up to $3.6\times$ lower latency, and remains competitive or faster across most parameter regimes. Compared to InstantOMR, StOMR is slower for very small bulletin boards but becomes substantially faster beyond 2^{11} transactions, with over $15\times$ speedup at 2^{16} . StOMR-GPU is a further $1.6\text{--}3.7\times$ faster than StOMR-CPU on the bulletin-board scaling sweep ($3.3\text{--}13.6\times$ over SophOMR, over $25\times$ over InstantOMR at 2^{16}), and the GPU's query-level batching yields a $17\times$ latency reduction at $P = 64$ versus a single-query design – outperforming every baseline across the full pertinent-message range we measure.

StOMR is particularly efficient in low-to-moderate numbers of pertinent messages, where reduced ciphertext expansion significantly lowers evaluation cost.

We further evaluate deployment overhead by measuring on-chain digest publication costs under Ethereum calldata pricing (16 gas per nonzero byte) and Zcash relay fees. Due to its compact 4-byte digest, StOMR reduces on-chain cost compared to prior work, yielding approximately $1.5\times$ – $3\times$ savings on Ethereum, where cost is dominated by fixed per-transaction gas overhead, two to three orders of magnitude (up to $667\times$) under Zcash fee models, where cost scales directly with payload size.

1.4 Related Work

1.4.1 Oblivious Message Retrieval (OMR). OMR was first introduced in [45] and enables anonymous messaging systems to outsource receiver-side computation to a powerful server while maintaining recipient privacy. Naturally, the group setting is central to anonymous messaging. Group OMR [46] extends OMR to support group messaging while improving efficiency over the naive approach, in which the detector’s workload grows linearly with the group size. PerfOMR [47] improved upon [45] by using RLWE instead of LWE for the signal generation. Next, SophOMR [41] introduced a new compression mechanism that fully exploits the native homomorphic SIMD structure of the underlying HE scheme and reduced both the runtime and digest size compared to PerfOMR. The recent work of [44] proves that prior OMR constructions are resistant against denial of service (DoS) attacks under a standard LWE assumption. While prior work primarily focuses on improving amortized cost and bandwidth through batching, InstantOMR [42] instead targets low latency and high parallelism using a two-layer bootstrapping architecture and a hybrid combination of TFHE and RLWE homomorphic operations. Its performance falls short of the state-of-the-art SophOMR, requiring 274.51 ms per message compared to 1.68 ms on a single CPU. However, the scheme parallelizes efficiently and approaches SophOMR in the 180-CPU experiment, achieving 1.69 ms per message versus 1.38 ms.

In an orthogonal line of work, HomeRun [39] introduces an OMR scheme on the two semi-honest and non-colluding server model. Finally, Two-Face [22] utilizes custom zero-knowledge proofs to achieve an actively secure private signaling protocol in the two non-colluding server model.

1.4.2 Fuzzy Message Detection (FMD) and Private Signaling. The work of Private Signaling [49] relies on trusted execution environments (TEE), which is a strong trust assumption. Following from [49], [38] leverages Oblivious RAM (ORAM) on TEEs to improve the computation complexity of the server. FMD [12, 31, 55] is similar to OMR as they allow a recipient to derive a specialized message detection key, but unfortunately, they also introduce false positives as they incorrectly identify non-matching messages.

1.4.3 Private Information Retrieval (PIR). In PIR [21], a client wants to retrieve an item from a server’s database without revealing to the server which item. Traditional PIR assumes that database items are addressed by index [2, 3, 8, 25, 26, 34, 63, 64]. In contrast, keyword PIR models the database as a key–value store, allowing the client

to retrieve the value associated with a keyword while keeping the queried keyword private [18, 33, 50, 54].

Single-client PIR schemes typically rely on a heavy preprocessing phase in which the client computes hints offline. When the database changes, this preprocessing must be repeated. Recent work, thus, focuses on updatable PIR [35, 43, 48], enabling the client to update its state in time proportional to the number of database updates.

The work of [9] constructs anonymous communication using label-based retrieval over a PIR-accessible database, where labels must be mapped to database indices to enable private lookup. This mapping introduces additional complexity in supporting general label spaces, often requiring auxiliary search structures and multi-round PIR-based lookup procedures that increase server-side processing overhead.

Although PIR and OMR may appear similar, they differ fundamentally. In PIR, the client issues a query for a specific index or label while hiding that identifier from the server. In contrast, OMR performs retrieval using the recipient’s evaluation key, which reveals nothing about the messages that may be retrieved. Moreover, PIR assumes the recipient already knows the indices of the desired messages, whereas OMR assumes recipients lack prior knowledge of message locations. Keyword PIR better matches this setting, but because the bulletin board updates frequently, the scheme must also support efficient updates. Furthermore, the receiver may need to retrieve multiple relevant messages, requiring batched queries. To the best of our knowledge, there is no efficient keyword, batched, and updatable PIR scheme to date.

Finally, designing stateful OMR schemes in the group setting [4, 5, 7], as well as incorporating additional features such as content moderation [6] and subversion-resilience [27], remain compelling directions for future work.

2 Preliminaries

2.1 Homomorphic Encryption

A homomorphic encryption (HE) scheme with respect to a circuit class $\mathcal{F}$, message space $\mathcal{M}$, and ciphertext space $\mathcal{C}$, is a tuple of probabilistic polynomial-time algorithms (KeyGen, Enc, Dec, Eval) described as below [16, 28, 30].

- $(sk, pk) \leftarrow \text{KeyGen}(1^\lambda)$: The key generation algorithm receives a security parameter λ and returns a secret key sk and public key pk .
- $ct \leftarrow \text{Enc}(pk, m)$: The encryption algorithm receives a public key pk and a message $m \in \mathcal{M}$, and returns a ciphertext $ct \in \mathcal{C}$.
- $m \leftarrow \text{Dec}(sk, ct)$: The decryption algorithm receives a secret key sk and a ciphertext $ct \in \mathcal{C}$, and returns a message m .
- $ct' \leftarrow \text{Eval}(pk, f, ct)$: The evaluation algorithm receives a public key pk , a circuit f , and a ciphertext $ct \in \mathcal{C}^*$, and produces an evaluation output ct' .

Correctness. An HE scheme with respect to a circuit class $\mathcal{F}$, message space $\mathcal{M}$, and ciphertext space $\mathcal{C}$, is correct, if for any

$f \in \mathcal{F}$ and $m \in \mathcal{M}^*$:

$$\Pr \left[\text{Dec}(\text{sk}, \text{Eval}(\text{pk}, f, \text{Enc}(\text{pk}, m))) = f(m) \mid (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda) \right] \geq 1 - \text{negl}(\lambda).$$

Definition 2.1 (HE Security). An HE scheme with message space $\mathcal{M}$, is secure, if for any (sk, pk) sampled according to $\text{KeyGen}(1^\lambda)$, and any $m_0, m_1 \in \mathcal{M}$, and any PPT adversary $\mathcal{A}$:

$$\left| \Pr \left[\mathcal{A}(\text{pk}, \text{ct}_0) = 1 \mid \text{ct}_0 \leftarrow \text{Enc}(\text{pk}, m_0) \right] - \Pr \left[\mathcal{A}(\text{pk}, \text{ct}_1) = 1 \mid \text{ct}_1 \leftarrow \text{Enc}(\text{pk}, m_1) \right] \right| \leq \text{negl}(\lambda).$$

BFV Scheme. The BFV scheme [16, 28] is a lattice-based homomorphic encryption scheme defined over the ring $R = \mathbb{Z}[x]/(x^N + 1)$, where N is a power-of-two. The ring dimension N is one of the key parameters that determine the security level. Ciphertexts are elements of the ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$, where q is the ciphertext modulus that controls the noise budget and therefore the supported homomorphic multiplicative depth. Plaintexts are elements of the ring $R_p = \mathbb{Z}_p[x]/(x^N + 1)$, where p is the plaintext modulus defining the message space. The BFV scheme encrypts integers modulo the plaintext modulus p and supports homomorphic single-instruction multiple-data (SIMD) arithmetic over $\mathbb{Z}_p$ [57]. When batching is enabled, the plaintext ring R_p decomposes via the Chinese Remainder Theorem into $R_p \cong \mathbb{Z}_p^N$ provided that $p \equiv 1 \pmod{2N}$. Under this condition, a single ciphertext can encode a vector of N messages in $\mathbb{Z}_p$, which are referred to as SIMD slots. Homomorphic addition and multiplication operate component-wise across these slots, enabling parallel computation on all packed messages. When interactions across slots are required, such as summing all slots, ciphertext rotations corresponding to ring automorphisms are used to permute the slots.

Batching packs multiple plaintexts into a single ciphertext:

$$\text{ct} \leftarrow \text{BatchEnc}(\text{pk}, \text{lst}),$$

where $\text{lst} = [m_1, \dots, m_N]$ is a vector of N plaintext messages in $\mathbb{Z}_p$ and ct is the resulting ciphertext encoding these values.

Recall that the homomorphic evaluation procedure $\text{Eval}(\text{pk}, f, \text{ct})$ operates on a function f . In BFV, the native arithmetic operations are addition, subtraction, and multiplication, which we denote by ADD, SUB, and MUL, respectively. We denote an equality of two ciphertexts by EQ and implement it using Fermat's Little Theorem $(\text{ct}_1 - \text{ct}_2)^{p-1} \pmod{p}$ for a prime p [32, 37]. Finally, we denote ciphertext slot rotations by ROT, which correspond to automorphisms of the ring R_q and cyclic permutations of the SIMD slots.

Homomorphic Compression. The goal of homomorphic compression [14, 19, 20, 29, 45] is to homomorphically compress an encryption of a sparse vector, which has only a few non-zero components, into an encryption of a shorter vector, which can later be efficiently decompressed after decryption. In the context of OMR, homomorphic compression is an important building block for ensuring that the communication cost does not scale linearly with the bulletin board size. This is achieved by applying homomorphic compression after performing homomorphic equality checks on the signal, the results of which the detector does not learn in plaintext.

In this work, we directly apply the homomorphic compression scheme used in SophOMR [41], which refines the scheme from [19],

to the OMR parameter setting. This approach offers high efficiency due to its compatibility with homomorphic SIMD operations, as the compression essentially consists of multiplying a plaintext Vandermonde-like matrix with an encrypted vector [40]. For a more in-depth discussion, please refer to [19, 41].

2.2 Pseudorandom Function Family (PRF)

A pseudorandom function family PRF is a keyed collection of efficiently computable functions $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^m$. We denote the function instantiated with k over input x as $\text{PRF}_k(x)$. The family PRF is pseudorandom if for every probabilistic polynomial-time adversary $\mathcal{A}$,

$$\left| \Pr_{k \leftarrow \{0, 1\}^\lambda} [\mathcal{A}^{\text{PRF}_k(\cdot)}(1^\lambda) = 1] - \Pr_{f \leftarrow \mathcal{F}_{n,m}} [\mathcal{A}^{f(\cdot)}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\mathcal{F}_{n,m}$ denotes the set of all functions from $\{0, 1\}^n$ to $\{0, 1\}^m$.

2.3 Oblivious Message Retrieval (OMR)

An Oblivious Message Retrieval (OMR) scheme (w.r.t. payload space $\mathcal{P}$) is a tuple of probabilistic algorithms (ParamGen, KeyGen, Signal, Digest, Decode) described below:

- $\text{pp} \leftarrow \text{ParamGen}(1^\lambda)$: The parameter generation algorithm takes a security parameter λ and returns public parameters pp . The following algorithms implicitly take pp as input.
- $(\text{sk}, \text{pk}_{\text{sig}}, \text{pk}_{\text{det}}) \leftarrow \text{KeyGen}(\text{pp})$: The key generation algorithm outputs a secret key sk , a signal key pk_{sig} , and a detection key pk_{det} .
- $\text{sig} \leftarrow \text{Signal}(\text{pk}_{\text{sig}})$: The signaling algorithm takes a signal key pk_{sig} and outputs a signal sig .
- $\text{dgst} \leftarrow \text{Digest}(\text{BB}, \text{pk}_{\text{det}}, P)$: The digesting algorithm takes a bulletin board $\text{BB} = ((x_1, \text{sig}_1), \dots, (x_N, \text{sig}_T))$ of size T , a detection key pk_{det} , and an upper bound P on the number of pertinent payloads addressed to the receiver. It outputs a digest dgst .
- $\text{msg} \leftarrow \text{Decode}(\text{dgst}, \text{sk})$: The decoding algorithm takes a digest dgst and a secret key sk . It outputs a list of decoded payloads $\text{msg} \in \mathcal{P}^*$.

The bulletin board $\text{BB} = ((x_1, \text{sig}_1), \dots, (x_T, \text{sig}_T))$ is formed by several parties uploading payloads $x_i \in \mathcal{P}$ tagged with signals sig_i with respect to the signal key of the intended receiver.

Correctness. Let $(\text{sk}, \text{pk}_{\text{sig}}, \text{pk}_{\text{det}}) \leftarrow \text{KeyGen}(\text{pp})$. Let BB be a bulletin board of length T and $\widetilde{\text{msg}}$ be the list of pertinent payloads in BB addressed to pk_{sig} . For $P \geq |\widetilde{\text{msg}}|$, the following holds:

$$\Pr \left[\text{msg} \neq \widetilde{\text{msg}} \mid \begin{array}{l} \text{dgst} \leftarrow \text{Digest}(\text{BB}, \text{pk}_{\text{det}}, P), \\ \text{msg} \leftarrow \text{Decode}(\text{dgst}, \text{sk}) \end{array} \right] \leq \text{negl}(\lambda).$$

Receiver privacy experiment. We define the receiver privacy experiment $\text{Exp}_{\text{RP},b}^{\text{OMR}}$ that is parameterized by an OMR scheme $\text{OMR} = (\text{ParamGen}, \text{KeyGen}, \text{Signal}, \text{Digest}, \text{Decode})$ and bit $b \in \{0, 1\}$. For any PPT adversary $\mathcal{A}$, $\text{Exp}_{\text{RP},b}^{\text{OMR}}(1^\lambda, \mathcal{A})$ is defined as follows:

$$\text{Exp}_{\text{RP},b}^{\text{OMR}}(1^\lambda, \mathcal{A}):$$

- $\text{pp} \leftarrow \text{ParamGen}(1^\lambda)$,
- $(\text{sk}_0, \text{pk}_{\text{sig}}^0, \text{pk}_{\text{det}}^0) \leftarrow \text{KeyGen}(\text{pp})$,
- $(\text{sk}_1, \text{pk}_{\text{sig}}^1, \text{pk}_{\text{det}}^1) \leftarrow \text{KeyGen}(\text{pp})$,

- $\text{sig}_0 \leftarrow \text{Signal}(\text{pk}_{\text{sig}}^0)$; $\text{sig}_1 \leftarrow \text{Signal}(\text{pk}_{\text{sig}}^1)$,
- $\text{aux} \leftarrow (\text{pp}, \text{pk}_{\text{sig}}^0, \text{pk}_{\text{det}}^0, \text{pk}_{\text{sig}}^1, \text{pk}_{\text{det}}^1)$,
- $b' \leftarrow \mathcal{A}(\text{aux}, \text{sig}_b)$,
- Return b' .

Definition 2.2 (Receiver Privacy of OMR). Let $\text{OMR} = (\text{ParamGen}, \text{KeyGen}, \text{Signal}, \text{Digest}, \text{Decode})$, be an OMR scheme. Then, OMR satisfies receiver privacy if for all, sufficiently large, λ , and any PPT adversary $\mathcal{A}$:

$$\left| \Pr[\text{Exp}_{\text{RP},0}^{\text{OMR}}(1^\lambda, \mathcal{A}) = 1] - \Pr[\text{Exp}_{\text{RP},1}^{\text{OMR}}(1^\lambda, \mathcal{A}) = 1] \right| \leq \text{negl}(\lambda).$$

2.4 Public-Key Encryption

A public-key encryption (PKE) scheme with message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ is a tuple of probabilistic algorithms (KeyGen , Enc , Dec) such that:

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$: The key generation algorithm takes the security parameter λ and outputs a public key pk and a secret key sk .
- $c \leftarrow \text{Enc}(\text{pk}, m)$: The encryption algorithm takes a public key pk and a message $m \in \mathcal{M}$ and outputs a ciphertext $c \in \mathcal{C}$.
- $m \leftarrow \text{Dec}(\text{sk}, c)$: The decryption algorithm takes a secret key sk and a ciphertext $c \in \mathcal{C}$ and outputs a message $m \in \mathcal{M}$ or a special symbol $\perp$.

Correctness. For all sufficiently large $\lambda \in \mathbb{N}$ and all $m \in \mathcal{M}$, the following holds:

$$\Pr \left[m \neq \text{Dec}(\text{sk}, c) \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ c \leftarrow \text{Enc}(\text{pk}, m) \end{array} \right] \leq \text{negl}(\lambda).$$

IND-CPA security. We define the IND-CPA experiment $\text{Exp}_{\text{IND-CPA},b}^{\text{PKE}}$ parameterized by a PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ and bit $b \in \{0, 1\}$. For any PPT adversary $\mathcal{A}$, $\text{Exp}_{\text{IND-CPA},b}^{\text{PKE}}(1^\lambda, \mathcal{A})$ is defined as follows:

$\text{Exp}_{\text{IND-CPA},b}^{\text{PKE}}(1^\lambda, \mathcal{A})$:

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$,
- $(m_0, m_1, \text{aux}) \leftarrow \mathcal{A}(\text{pk})$,
- $c \leftarrow \text{Enc}(\text{pk}, m_b)$,
- $b' \leftarrow \mathcal{A}(\text{aux}, c)$,
- Return b' .

Definition 2.3 (IND-CPA security of PKE). A PKE scheme PKE is IND-CPA secure if for all PPT adversaries $\mathcal{A}$

$$\left| \Pr[\text{Exp}_{\text{IND-CPA},0}^{\text{PKE}}(1^\lambda, \mathcal{A}) = 1] - \Pr[\text{Exp}_{\text{IND-CPA},1}^{\text{PKE}}(1^\lambda, \mathcal{A}) = 1] \right| \leq \text{negl}(\lambda).$$

Receiver privacy experiment. We define the receiver anonymity experiment $\text{Exp}_{\text{RP},b}^{\text{PKE}}$, parameterized by a PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ and bit $b \in \{0, 1\}$. For any PPT adversary $\mathcal{A}$, $\text{Exp}_{\text{RP},b}^{\text{PKE}}(1^\lambda, \mathcal{A})$ is defined as follows:

$\text{Exp}_{\text{RP},b}^{\text{PKE}}(1^\lambda, \mathcal{A})$:

- $(\text{pk}_0, \text{sk}_0) \leftarrow \text{KeyGen}(1^\lambda)$,
- $(\text{pk}_1, \text{sk}_1) \leftarrow \text{KeyGen}(1^\lambda)$,
- $(m, \text{aux}) \leftarrow \mathcal{A}(\text{pk}_0, \text{pk}_1)$,

- $c \leftarrow \text{Enc}(\text{pk}_b, m)$,
- $b' \leftarrow \mathcal{A}(\text{aux}, c)$,
- Return b' .

Definition 2.4 (Receiver privacy security of PKE). A PKE scheme PKE satisfies receiver privacy if for all PPT adversaries $\mathcal{A}$

$$\left| \Pr[\text{Exp}_{\text{RP},0}^{\text{PKE}}(1^\lambda, \mathcal{A}) = 1] - \Pr[\text{Exp}_{\text{RP},1}^{\text{PKE}}(1^\lambda, \mathcal{A}) = 1] \right| \leq \text{negl}(\lambda).$$

3 OMR Key Encapsulation Mechanism

In this section we present the notion of OMR-KEM.

3.1 OMR-KEM Overview

In Fig. 1, we show an overview of OMR-KEM. More specifically, for each entity, we show which steps it performs and how they differ from S-OMR. Both protocols start with parameter generation in step ①, parameterized by a security parameter λ . The output of the parameter generation is published and used by the sender(s) and the receiver(s). After the first step, the two protocols diverge, so we describe one at a time.

The purpose of OMR-KEM is to enable a sender and a receiver to derive a shared state without any prior communication. If an out-of-band channel already exists between the two parties, then OMR-KEM can be omitted. Below, we delve into the next steps of OMR-KEM from Fig. 1.

② **Key Generation:** The receiver generates a symmetric key and a public key for OMR-KEM, as well as a detection key pk_{det} . The public key pk is sent to the sender, as in any KEM, and pk_{det} is sent to the detector.

③ **Encapsulation:** The sender uses the public key pk to generate a symmetric key k and a ciphertext $\text{ct} = (x, \text{sig})$, where x encrypts k and sig is a detection signal, similar to those in [41, 42]. Note that ct is the encapsulation of k along with a signal that makes it detectable. The sender posts (x, sig) to the bulletin board and goes offline.

④ **Digesting:** Upon a receiver's request, the detector uses pk_{det} to scan the bulletin board for pertinent messages.¹ The digest algorithm is parameterized by an upper bound P on the number of such messages. The detector compresses them into a digest dgst and sends it to the receiver. Crucially, the detector learns nothing about the receiver's messages, including whether any exist.

⑤ **Decapsulation:** Finally, the receiver runs decapsulation and retrieves a list of keys keys , potentially from multiple senders.

3.2 OMR-KEM Definition

An OMR-based Key Encapsulation Mechanism (OMR-KEM) (w.r.t. key space $\mathcal{K}$) is a tuple of probabilistic algorithms (ParamGen , KeyGen , Encap , Digest , Decap) described below:

- $\text{pp} \leftarrow \text{ParamGen}(1^\lambda)$: The parameter generation algorithm takes a security parameter λ and returns public parameters pp . The following algorithms implicitly take pp as input.
- $(\text{sk}, \text{pk}, \text{pk}_{\text{det}}) \leftarrow \text{KeyGen}(\text{pp})$: The key generation algorithm outputs a secret key sk , a public key pk , and a detection key pk_{det} .

¹"Pertinent messages" are payloads addressed to the receiver.

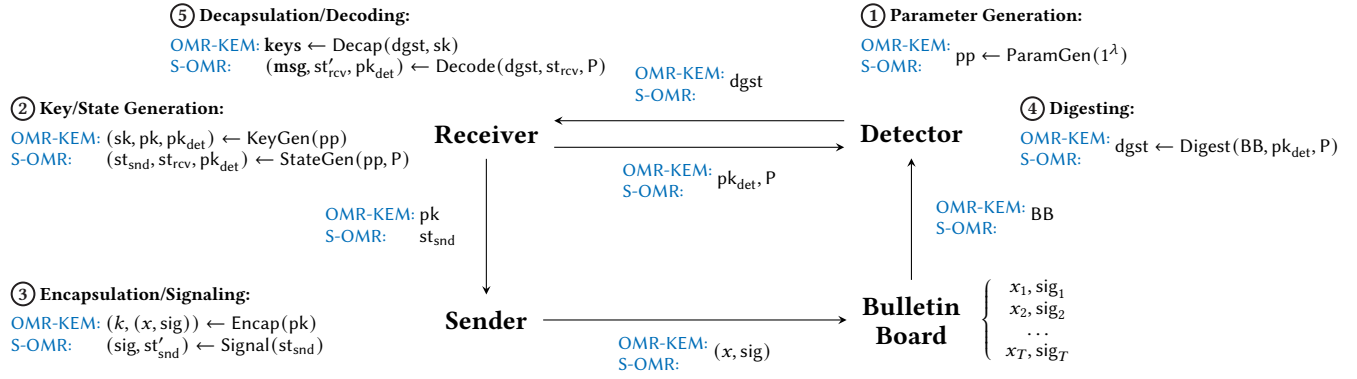


Figure 1: Overview of OMR-based key encapsulation (OMR-KEM) mechanism and stateful OMR (S-OMR). x denotes the payload submitted by the sender. For OMR-KEM, this payload is fixed to be the encryption of the key k , while in S-OMR, we leave it unspecified and it can be any arbitrary bitstring. Finally, P denotes the pertinent messages upper bound.

- $(k, ct) \leftarrow \text{Encap}(pk)$: The encapsulation algorithm takes a public key pk and outputs a key k and a ciphertext ct .
- $dgst \leftarrow \text{Digest}(BB, pk_{det}, P)$: The digesting algorithm takes a bulletin board $BB = (ct_1, \dots, ct_T)$ of size T , a detection key pk_{det} , and an upper bound P on the number of pertinent payloads addressed to the receiver. It outputs a digest $dgst$.
- $keys \leftarrow \text{Decap}(dgst, sk)$: The decapsulation algorithm takes a digest $dgst$ and a secret key sk . It outputs a list of decapsulated keys $keys \in \mathcal{K}^*$.

The bulletin board $BB = (ct_1, \dots, ct_T)$ is formed by several parties uploading ciphertexts with respect to the encapsulation key of the intended receiver.

Correctness. Let $(sk, pk, pk_{det}) \leftarrow \text{KeyGen}(pp)$. Let BB be a bulletin board of length T and let

$$\widetilde{keys} := (k_i \mid i \in [T], ct_i \in BB, (k_i, ct_i) \text{ honestly generated w.r.t. } pk).$$

That is, $\widetilde{keys}$ is the list of keys generated by honest executions of Encap over pk and the corresponding ciphertexts were stored in BB . For $P \geq |\widetilde{keys}|$, the following holds:

$$\Pr \left[\widetilde{keys} \neq keys \mid \begin{array}{l} dgst \leftarrow \text{Digest}(BB, pk_{det}, P), \\ keys \leftarrow \text{Decap}(dgst, sk) \end{array} \right] \leq \text{negl}(\lambda).$$

Receiver privacy for OMR-KEM is along the lines of Section 2.3, where the Signal operation has been substituted by Encap , the message space $\mathcal{M}$ is substituted by the key space $\mathcal{K}$, and the adversary receives the challenge ciphertext ct_b output by Encap .

IND-CPA Security of OMR-KEM. Let $\text{OMR-KEM} = (\text{ParamGen}, \text{KeyGen}, \text{Encap}, \text{Digest}, \text{Decap})$ be an OMR-based KEM with key space $\mathcal{K}$. We define the IND-CPA experiment $\text{Exp}_{\text{IND-CPA},b}^{\text{OMR-KEM}}$ parameterized by bit $b \in \{0, 1\}$. For any PPT adversary $\mathcal{A}$, $\text{Exp}_{\text{IND-CPA},b}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A})$ is defined as follows:

$$\text{Exp}_{\text{IND-CPA},b}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A}):$$

- $pp \leftarrow \text{ParamGen}(1^\lambda)$,
- $(sk, pk, pk_{det}) \leftarrow \text{KeyGen}(pp)$,
- $(k_0, ct) \leftarrow \text{Encap}(pk)$,
- $k_1 \leftarrow \mathcal{K}$,

- $aux \leftarrow (pp, pk, pk_{det}, ct)$,
- $b' \leftarrow \mathcal{A}(aux, k_b)$,
- Return b' .

Definition 3.1 (IND-CPA Security of OMR-based KEM). Let $\text{OMR-KEM} = (\text{ParamGen}, \text{KeyGen}, \text{Encap}, \text{Digest}, \text{Decap})$ be an OMR-based KEM. We say that OMR-KEM is IND-CPA secure if for all sufficiently large λ and any PPT adversary $\mathcal{A}$:

$$\left| \Pr[\text{Exp}_{\text{IND-CPA},0}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A}) = 1] - \Pr[\text{Exp}_{\text{IND-CPA},1}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A}) = 1] \right| \leq \text{negl}(\lambda).$$

4 Stateful OMR (S-OMR)

In this section we present the notion of stateful OMR.

4.1 S-OMR Overview

S-OMR enables senders to post compact signals on the bulletin board while allowing receivers to privately recover their messages. Unlike OMR-KEM, which establishes shared key material between two parties, S-OMR assumes that the sender and receiver already share a secret state. This state evolves as messages are sent and retrieved, enabling the use of lightweight symmetric cryptography instead of public-key encryption.

① **Parameter Generation:** The detector runs the parameter generation algorithm based on a security parameter λ and returns a public parameter set used by the sender and the receiver.

② **State Generation:** The receiver runs the state generation algorithm to initialize the protocol. The algorithm outputs a sender's state st_{snd} , a receiver's state st_{rcv} , and a detection key pk_{det} . The receiver sends st_{snd} to the sender and pk_{det} to the detector, while retaining st_{rcv} . The sender's state enables the sender to generate signals for future messages, whereas the receiver's state allows the receiver to decode digests returned by the detector.

③ **Signaling:** To send a message, the sender runs the signaling algorithm on its current state st_{snd} . The algorithm outputs a signal sig and an updated state st'_{snd} . The sender then posts the entry (x, sig) to the bulletin board, where x denotes the payload intended for the receiver. After posting, the sender replaces st_{snd} with st'_{snd} .

④ **Digesting**: When the receiver wishes to retrieve messages, it requests a digest from the detector. Using the detection key pk_{det} , the detector scans the bulletin board entries (x_i, sig_i) and, given an upper bound P on the number of pertinent messages, compresses the relevant ones into a digest $dgst$. The detector returns $dgst$ to the receiver without learning which entries correspond to it.

⑤ **Decoding**: Finally, the receiver runs the decoding algorithm on $dgst$ using its state st_{rcv} . The algorithm outputs the list of recovered payloads msg and an updated state st'_{rcv} , which is used for subsequent retrievals, as well as an updated detection key pk_{det} .

4.2 S-OMR Definition

A stateful Oblivious Message Retrieval (S-OMR) scheme (w.r.t. payload space $\mathcal{P}$) is a tuple of probabilistic algorithms (ParamGen, StateGen, Signal, Digest, Decode) described below:

- $pp \leftarrow \text{ParamGen}(1^\lambda)$: The parameter generation algorithm takes a security parameter λ and returns public parameters pp . The following algorithms implicitly take pp as input.
- $(st_{snd}, st_{rcv}, pk_{det}) \leftarrow \text{StateGen}(pp, P)$: The state generation algorithm receives the public parameters pp and the number of pertinent messages, and outputs a sender's state st_{snd} , a receiver's state st_{rcv} , and a detection key pk_{det} .
- $(sig, st'_{snd}) \leftarrow \text{Signal}(st_{snd})$: The signaling algorithm takes a sender's state st_{snd} and outputs a signal sig and a new sender's state st'_{snd} .
- $dgst \leftarrow \text{Digest}(BB, pk_{det}, P)$: The digesting algorithm takes a bulletin board $BB = ((x_1, sig_1), \dots, (x_T, sig_T))$ of size T , a detection key pk_{det} , and an upper bound P on the number of pertinent payloads addressed to the receiver. It outputs a digest $dgst$.
- $(msg, st'_{rcv}, pk_{det}) \leftarrow \text{Decode}(dgst, st_{rcv}, P)$: The decoding algorithm takes a digest $dgst$, a receiver's state st_{rcv} , and the number of pertinent messages P . It outputs a list of decoded payloads $msg \in \mathcal{P}^*$, a new receiver's state st'_{rcv} and the updated detection key pk_{det} .

Correctness. Let $(st_{snd}^0, st_{rcv}^0, pk_{det}^0) \leftarrow \text{StateGen}(pp, P)$. For any $T \in \mathbb{N}$, consider any sequence of honestly generated signals

$$(sig_i, st_{snd}^{i-1}) \leftarrow \text{Signal}(st_{snd}^{i-1}), \quad i \in [T],$$

and let $BB = ((x_1, sig_1), \dots, (x_T, sig_T))$ be a bulletin board containing these pairs (and possibly additional arbitrary entries).

Let $\widetilde{msg}$ be the list of payloads x_i whose signals sig_i were honestly generated from the evolving sender's state st_{snd} .

For any $P \geq |\widetilde{msg}|$, the following holds:

$$\Pr \left[msg \neq \widetilde{msg} \mid \begin{array}{l} dgst \leftarrow \text{Digest}(BB, pk_{det}, P), \\ (msg, st'_{rcv}, pk_{det}) \leftarrow \text{Decode}(dgst, st_{rcv}^0, P) \end{array} \right] \leq \text{negl}(\lambda).$$

Receiver Privacy Experiment. We define the receiver privacy experiment $\text{Exp}_{RP,b}^{\text{S-OMR}}$, parameterized by a stateful OMR scheme $\text{S-OMR} = (\text{ParamGen}, \text{StateGen}, \text{Signal}, \text{Digest}, \text{Decode})$ and bit $b \in \{0, 1\}$. For any PPT adversary $\mathcal{A}$, $\text{Exp}_{RP,b}^{\text{S-OMR}}(1^\lambda, \mathcal{A})$ is defined as follows:

$$\text{Exp}_{RP,b}^{\text{S-OMR}}(1^\lambda, \mathcal{A}):$$

- $pp \leftarrow \text{ParamGen}(1^\lambda)$,

- $(st_{snd}^0, st_{rcv}^0, pk_{det}^0) \leftarrow \text{StateGen}(pp, P)$,
- $(st_{snd}^1, st_{rcv}^1, pk_{det}^1) \leftarrow \text{StateGen}(pp, P)$,
- $aux \leftarrow (pp, pk_{det}^0, pk_{det}^1)$,
- $b' \leftarrow \mathcal{A}^{O_{\text{Sig}}(st_{snd}^b), O_{\text{Dec}}(st_{rcv}^0, st_{rcv}^1, P, \cdot)}(aux)$,
- Return b' ,

where $O_{\text{Sig}}(st_{snd}^b)$ denotes a stateful Signal oracle that on each query computes as $(sig, st_{snd}^b) \leftarrow \text{Signal}(st_{snd}^b)$, and returns sig , and $O_{\text{Dec}}(st_{rcv}^0, st_{rcv}^1, P, \cdot)$ is a stateful oracle that on query $dgst$, computes $(\cdot, st_{rcv}^i, pk_{det}^i) \leftarrow \text{Decode}(dgst, st_{rcv}^i, P)$, and returns pk_{det}^i , for $i \in \{0, 1\}$.

Definition 4.1 (Receiver Privacy of S-OMR). The scheme satisfies receiver privacy if for all sufficiently large λ and all PPT adversaries $\mathcal{A}$:

$$\left| \Pr[\text{Exp}_{RP,0}^{\text{S-OMR}}(1^\lambda, \mathcal{A}) = 1] - \Pr[\text{Exp}_{RP,1}^{\text{S-OMR}}(1^\lambda, \mathcal{A}) = 1] \right| \leq \text{negl}(\lambda).$$

5 Our Protocols

5.1 FHE-Based Stateful OMR Construction

In the current section, we present our FHE-based S-OMR scheme, depicted in Fig. 2.

The parameter generation operation of the FHE-based stateful OMR scheme, $pp \leftarrow \text{S-OMR.ParamGen}(1^\lambda)$, outputs public parameters pp , that include parameters for the underlying FHE system, namely the polynomial degree, size of the coefficient modulus q , and plaintext modulus p (which collectively impact the maximum depth of computation, the number of slots available, and the security level). The state generation operation, $(st_{snd}, st_{rcv}, pk_{det}) \leftarrow \text{S-OMR.StateGen}(pp, P)$, samples a key $k \leftarrow \mathcal{K}$ and initializes the sender's state as $st_{snd} = (1, k)$. Then, generates an FHE key pair $(pk_{fhe}, sk_{fhe}) \leftarrow \text{FHE.KeyGen}(1^\lambda)$ and initializes the receiver's state as $st_{rcv} = (pk_{fhe}, sk_{fhe}, 1, k)$. The detection key pk_{det} is initialized with the public key pk_{fhe} and includes a batched encryption of the first P signals, enabling the receiver to later identify relevant signals on the bulletin board.

The signal operation, $(sig, st_{snd}) \leftarrow \text{S-OMR.Signal}(st_{snd})$, takes the current sender state (i, k) , computes the next signal as $sig = \text{PRF}_k(i)$, increments the counter i , and updates the state to $st_{snd} = (i, k)$. The digest operation, $dgst \leftarrow \text{S-OMR.Digest}(BB, pk_{det}, P)$, takes a bulletin board BB of signals and partitions them into batches according to the number of slots per PRF. Each batch is processed under FHE to perform encrypted comparisons and logical operations, producing a pertinent vector pv . Finally, the digest $dgst$ is computed as a compressed representation of the BB payloads.

The decoding operation, $(msg, st_{rcv}, pk_{det}) \leftarrow \text{S-OMR.Decode}(dgst, st_{rcv}, P)$, uses the receiver's state $(pk_{fhe}, sk_{fhe}, i, k)$ to decrypt the digest $dgst$ with $\text{FHE.Dec}(sk_{fhe}, dgst)$ and recover a list of payloads $(v_1, \dots, v_p)$. The receiver determines the first unassigned index j such that $v_j = \perp$, extracts the decoded messages $msg = (v_1, \dots, v_{j-1})$, and updates the state index to $i \leftarrow i + j$. The detection key pk_{det} is updated with the next P signals generated via $lst = [\text{PRF}_k(j)]_{j \in [i, i+P]}$ and batched using FHE.BatchEnc . Finally, the receiver state is updated to $st_{rcv} = (pk_{fhe}, sk_{fhe}, i, k)$.

5.1.1 Batching. To amortize homomorphic computation, both the receiver's query and the sender's signal database are packed across

```

pp ← S-OMR.ParamGen( $1^\lambda$ ) :
1. Let  $p$  be the plaintext modulus,  $q$  be the ciphertext modulus, and  $N$ 
   be the polynomial degree. ▷ BGV parameters.
2. Return pp ← [ $p, q, N$ ]

( $st_{snd}, st_{rcv}, pk_{det}$ ) ← S-OMR.StateGen(pp, P) :
1.  $k \leftarrow \mathcal{K}$  ▷ Sample a key.
2.  $st_{snd} \leftarrow (1, k)$  ▷ Prepare the sender's state.
3.  $(pk_{fhe}, sk_{fhe}) \leftarrow \text{FHE.KeyGen}(1^\lambda)$ 
4.  $pk_{det} \leftarrow [pk_{fhe}]$ 
5.  $lst \leftarrow [PRF_k(i)]_{i \in [P]}$  ▷ Compute  $P$  signals based on state.
6.  $pk_{det}.append(\text{FHE.BatchEnc}(pk_{fhe}, lst, P))$  ▷ Encrypt (Sec. 5.1.1).
7.  $st_{rcv} \leftarrow (pk_{fhe}, sk_{fhe}, 1, k)$ 
8. Return  $(st_{snd}, st_{rcv}, pk_{det})$ 

(sig,  $st_{snd}$ ) ← S-OMR.Signal( $st_{snd}$ ) :
1.  $(i, k) \leftarrow st_{snd}$  ▷ Parse sender's state.
2.  $sig \leftarrow PRF_k(i)$  ▷ Compute the signal based on the state.
3.  $i \leftarrow i + 1$  ▷ Advance to the next state.
4.  $st_{snd} \leftarrow (i, k)$  ▷ Update sender's state.
5. Return  $(sig, st_{snd})$ 

dgst ← S-OMR.Digest(BB,  $pk_{det}, P$ ) :
1.  $[pk_{fhe}, q] \leftarrow pk_{det}$ 
2.  $bs \leftarrow N/(P * s)$  ▷ Batch size.  $N$  is the total number of slots,  $P$  is the
   number of pertinent messages, and  $s$  is the number of slots each PRF
   occupies.
3. Partition BB signals into  $lst \leftarrow (lst_1, \dots, lst_{\lceil BB/bs \rceil})$  ▷ Ignore pay-
   loads.
4.  $b \leftarrow [\text{FHE.BatchEnc}(pk_{fhe}, l, P)]_{l \in lst}$  ▷ Batched ctxts (Sec. 5.1.1).
5.  $pv \leftarrow []$  ▷ Empty pertinent vector.
6. For  $ct \in b$ : ▷ For each batched ciphertext.
   1.  $eq \leftarrow \text{FHE.Eval}(pk_{fhe}, EQ, ct, q)$  ▷ 0 if  $ct = q$ , else 1 (Sec 2.1).
   2.  $neq \leftarrow \text{FHE.Eval}(pk_{fhe}, SUB, 1, eq)$  ▷ Negate.
   3. For  $j \leftarrow s/2, s/4, \dots, 1$ : ▷ AND Tree: Rotate log  $s$  times
       1.  $tmp \leftarrow \text{FHE.Eval}(pk_{fhe}, ROT, neq, j)$  ▷ Rotate by  $j$ .
       2.  $neq \leftarrow \text{FHE.Eval}(pk_{fhe}, MUL, neq, tmp)$  ▷  $neq \wedge tmp$ .
   4.  $pv.append(neq)$  ▷  $neq = 1$  if all  $s$  bits are 1.
7. Return  $dgst \leftarrow \text{FHE.Eval}(pk_{fhe}, CMPS, BB.x, pv)$  ▷ Compress pay-
   loads from BB (Sec 2.1).

(msg,  $st_{rcv}, pk_{det}$ ) ← S-OMR.Decode(dgst,  $st_{rcv}, P$ ) :
1.  $(pk_{fhe}, sk_{fhe}, i, k) \leftarrow st_{rcv}$ 
2.  $[v_1, \dots, v_P] \leftarrow \text{FHE.Dec}(sk_{fhe}, dgst)$ 
3.  $j \leftarrow \min\{w \mid v_w = \perp\}; i \leftarrow i + j$  ▷ Find minimum index.
4.  $msg \leftarrow [v_1, \dots, v_{j-1}]$  ▷ Decoded message.
5.  $pk_{det} \leftarrow [pk_{fhe}]$  ▷ Next queries list.
6.  $lst \leftarrow [PRF_k(j)]_{j \in [i, \dots, i+P]}$  ▷ Compute next  $P$  signals.
7.  $pk_{det}.append(\text{FHE.BatchEnc}(pk_{fhe}, lst, P))$  ▷ Encrypt (Sec. 5.1.1).
8.  $st_{rcv} \leftarrow (pk_{fhe}, sk_{fhe}, i, k)$  ▷ Update receiver's state.
9. Return  $(msg, st_{rcv}, pk_{det})$ 

```

Figure 2: FHE-Based Stateful OMR (S-OMR) construction.

the SIMD slots of the underlying BGV ciphertexts. Let m denote the number of bits in our PRF output, i.e., the size of the signals in BB. Each BGV ciphertext provides N slots (Section 2.1). Each slot can encode up to $\lfloor \log_2(p) \rfloor$ bits, where p is the plaintext modulus. Thus, a single PRF output occupies $s = \lceil m / \lfloor \log_2 p \rfloor \rceil$ slots.

Intuitively, each signal PRF is represented as a block of s consecutive slots within a ciphertext. However, the batching layout

differs for the receiver's query and for the sender's signal list. Fig. 3 illustrates the slot layout for two example batching parameters P . The query ciphertext q , created by the receiver, replicates P identifiers across the slots so that it can be compared in parallel against many packed identifiers. Let P denote the number of pertinent matches expected by the receiver.² In contrast, the batched database ciphertexts pack multiple signal identifiers sequentially across the slot vector. This asymmetric layout allows a single homomorphic comparison to test the query against many candidate identifiers simultaneously.

Notice that each identifier occupies s slots (dotted lines in Fig. 3); thus, the number of identifiers that can be packed in a ciphertext while reserving space for the pertinent outputs is $b = \frac{N}{P \cdot s}$. We refer to b as the batch size. The sender partitions the identifiers in BB into groups $lst = (lst_1, \dots, lst_{\lceil BB/b \rceil})$, where each group contains at most b identifiers and is then packed into a ciphertext as:³

$$ct \leftarrow \text{FHE.BatchEnc}(pk_{fhe}, lst_i, P).$$

The result is a list of batched ciphertexts b , each encoding multiple identifiers arranged in contiguous blocks of s slots.

This slot structure enables the equality test in our protocol. After evaluating the slot-wise comparison between a batched ciphertext and the query, the protocol performs a rotation-and-multiplication reduction across the s slots of each identifier block. This reduction forms an AND-tree over the slots of each identifier, resulting in a single indicator bit per identifier that signals whether the corresponding identifier matches the query. These indicators form the pertinent vector used in the final compression step. Our compression relies on SophOMR [41] and is presented in Section 2.1.

5.1.2 Correctness and Security of FHE-based S-OMR.

THEOREM 5.1. *The S-OMR scheme of Fig. 2, S-OMR, satisfies correctness, assuming the correctness property of FHE and the pseudorandomness of PRF.*

Due to space limitations the proof of the above theorem is deferred for Section C.1.

THEOREM 5.2. *The S-OMR scheme of Fig. 2, satisfies receiver privacy, assuming the IND-CPA security of the underlying FHE scheme and the pseudorandomness property of PRF.*

PROOF. Our proof is via a sequence of hybrids.

Hybrid Hyb_0 . This hybrid matches $\text{Exp}_{\text{RP},0}^{\text{S-OMR}}(1^\lambda, \mathcal{A})$.

Hybrid Hyb_1^i . Let $rt_{\mathcal{A}}$ be an upper bound on the number of oracle queries made by $\mathcal{A}$. We define $\text{Hyb}_1^0 := \text{Hyb}_0$ and for $i \in [rt_{\mathcal{A}}]$, Hyb_1^i , in the first i computations where the protocol defines pk_{det} via $pk_{det}.append(\text{FHE.BatchEnc}(pk_{fhe}, lst, P))$, instead of using $lst \leftarrow [PRF_k(i)]_{i \in [P]}$, as the protocol dictates, the hybrid uses $lst \leftarrow [0^m]_{i \in [P]}$, i.e., instead of using FHE to batch encrypt PRF outputs, it batch encrypts zeros. We define $\text{Hyb}_1 := \text{Hyb}_1^{rt_{\mathcal{A}}}$.

²Notably, as PRFs are parameterized by a symmetric key k , different identifiers need to be used for different senders. For simplicity of presentation, our protocol treats P as if all the pertinent messages are from the same sender, but it also supports multiple senders.

³In the protocol description, we overload the FHE.BatchEnc operation and use the same notation for both client and server batching. The differences between client and server batching are explained in Section 5.1.1.

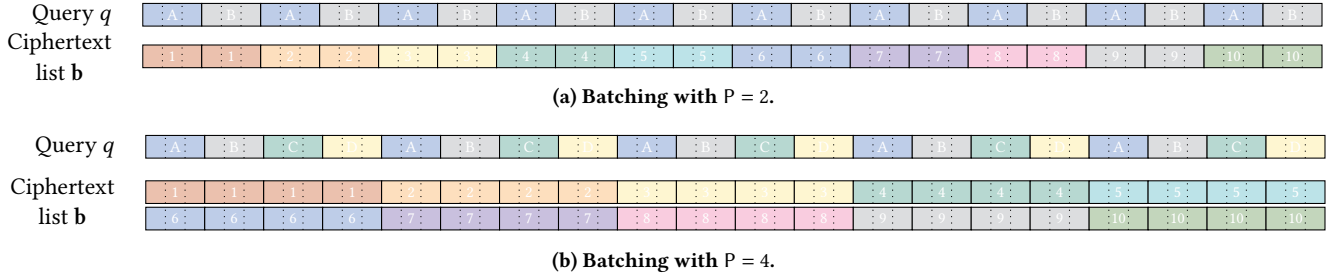


Figure 3: Batching example for $P = 2$ and $P = 4$ with $|\text{BB}| = 10$, total ciphertext slots $N = 60$, and slots per signal $s = 3$. The ciphertext list b is the batched version of BB , while the query q is the client's batched query. The structure of both b and q depends on P .

The indistinguishability between Hyb_1^i and Hyb_1^{i+1} follows by the IND-CPA security of FHE.

In more detail, for $i \in [\text{rt}_{\mathcal{A}}]$, we define an adversary $\mathcal{A}_i$ that plays against the IND-CPA security game of FHE. $\mathcal{A}_i$ receives pk_{fhe} from the challenger of FHE and for the entire computation before reaching the i -th $\text{pk}_{\text{det}}.\text{append}(\text{FHE}.\text{BatchEnc}(\text{pk}_{\text{fhe}}, \text{lst}, P))$ call, it fully simulates Hyb_1^{i-1} . When it reaches that specific call, it sets $\text{lst}_0 \leftarrow [\text{PRF}_k(i)]_{i \in [P]}$ as the protocol dictates, sets $\text{lst}_1 \leftarrow [0^m]_{i \in [P]}$, and sends $\text{lst}_0, \text{lst}_1$, to the challenger of the IND-CPA security game, and uses the response for the rest of the execution. Finally, outputs the bit b output by $\mathcal{A}$.

Observe that, if the challenger encrypts lst_0 , then $\mathcal{A}_i$ simulates perfectly Hyb_1^{i-1} for $\mathcal{A}$, otherwise it simulates perfectly Hyb_1^i , and the distinguishing advantage is bounded by the advantage of breaking the IND-CPA security of FHE.

Hybrid Hyb_2 . The current hybrid, simulates a uniformly random function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$, and instead of computing signals as $\text{sig} \leftarrow \text{PRF}_k(i)$, it computes $f(i)$. The indistinguishability between the hybrids follows from the pseudorandomness property of PRF. In more detail, we define an adversary $\mathcal{A}'$ that plays against the challenger of the pseudorandomness property of the PRF, and when the execution requires a PRF evaluation, it forwards the call to the challenger, and uses the response for the rest of the execution. Clearly, if the challenger returns PRF evaluations, then $\mathcal{A}'$ simulates perfectly Hyb_1 , otherwise, it perfectly simulates Hyb_2 , and the distinguishing advantage is bounded by the security of PRF.

Hybrid Hyb_3 . The current hybrid computes signals using the PRF key of st_{snd}^1 . The indistinguishability between the hybrids follows from the pseudorandomness property of PRF. In more detail, we define an adversary $\mathcal{A}'$ that plays against the challenger of the pseudorandomness property of the PRF, and whenever the execution requires a PRF evaluation, it simply forwards the call to the challenger, and uses the response for the rest of the execution. Clearly, if the challenger returns PRF evaluations, then $\mathcal{A}'$ simulates perfectly Hyb_3 , otherwise, it perfectly simulates Hyb_2 , and the distinguishing advantage is bounded by the security of PRF.

Hybrid Hyb_4^i . We define $\text{Hyb}_4^0 := \text{Hyb}_3$ and for $i \in [\text{rt}_{\mathcal{A}}]$, Hyb_4^i , in the first i computations where the protocol defines pk_{det} via $\text{pk}_{\text{det}}.\text{append}(\text{FHE}.\text{BatchEnc}(\text{pk}_{\text{fhe}}, \text{lst}, P))$, instead of using $\text{lst} \leftarrow [0^m]_{i \in [P]}$, it uses $\text{lst} \leftarrow [\text{PRF}_k(i)]_{i \in [P]}$, as the protocol dictates. We

define $\text{Hyb}_4 := \text{Hyb}_4^{\text{rt}_{\mathcal{A}}}$. The indistinguishability between Hyb_4^i and Hyb_4^{i+1} follows by the IND-CPA security of FHE.

In more detail, for $i \in [\text{rt}]$, we define an adversary $\mathcal{A}_i$ that plays against the IND-CPA security game of FHE. Adv' receives pk_{fhe} from the challenger of FHE and for the entire computation before reaching the i -th $\text{pk}_{\text{det}}.\text{append}(\text{FHE}.\text{BatchEnc}(\text{pk}_{\text{fhe}}, \text{lst}, P))$ call, it fully simulates Hyb_4^{i-1} . When it reaches that specific call, it sets $\text{lst}_0 \leftarrow [\text{PRF}_k(i)]_{i \in [P]}$ as the protocol dictates, sets $\text{lst}_1 \leftarrow [0^m]_{i \in [P]}$, and sends $\text{lst}_0, \text{lst}_1$, to the challenger of the IND-CPA security game, and uses the response for the rest of the execution. Finally, outputs the bit b output by $\mathcal{A}$.

Observe that, if the challenger encrypts lst_0 , then $\mathcal{A}_i$ simulates perfectly Hyb_4^i for $\mathcal{A}$, otherwise it simulates perfectly Hyb_4^{i-1} , and the distinguishing advantage is bounded by the advantage of breaking the IND-CPA security of FHE. $\square$

5.2 OMR-KEM Construction

In this section, we present our OMR-KEM construction, depicted in Fig. 4.

Let $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a public key encryption scheme with ciphertext space C and let $\text{OMR} = (\text{ParamGen}, \text{KeyGen}, \text{Signal}, \text{Digest}, \text{Decode})$ be an Oblivious Message Retrieval scheme with payload space $\mathcal{P} := C$. We define an OMR Key Encapsulation Mechanism scheme $\text{OMR-KEM} = (\text{ParamGen}, \text{KeyGen}, \text{Encap}, \text{Digest}, \text{Decap})$ (w.r.t. key space $\mathcal{K}$) in Fig. 4.

The ParamGen operation of OMR-KEM executes the corresponding operation of OMR. Next KeyGen , executes KeyGen of OMR and PKE, and pairs together the public and secret keys to form the public key and secret key of OMR-KEM. The encapsulation operation Encap computes a signal sig via a call to $\text{OMR}.\text{Signal}(\text{pk})$, samples a fresh key k , and generates a ciphertext c under PKE. The operation returns the key k and the ciphertext (c, sig) . The Digest operation of OMR-KEM executes the corresponding operation of OMR. Finally, the decapsulation operation executes the decoding procedure of OMR to recover the pertinent payloads msg , and then the decryption operation of PKE to recover the list of keys keys .

5.2.1 Security of OMR-KEM. In the current section, we prove correctness and security of the OMR-KEM scheme depicted in Fig. 4.

THEOREM 5.3. *The OMR-KEM scheme of Fig. 4, OMR-KEM, satisfies correctness, assuming the correctness property of the underlying*

```

pp ← OMR-KEM.ParamGen( $1^\lambda$ ) :
1. pp ← OMR.ParamGen( $1^\lambda$ )           ▷ OMR parameter generation.
2. Return pp

(sk, pk, pkdet) ← OMR-KEM.KeyGen(pp) :
1. (skomr, pksig, pkdet) ← OMR.KeyGen(pp)   ▷ OMR key generation.
2. (pkpke, skpke) ← PKE.KeyGen             ▷ PKE key generation.
3. Return sk := (skomr, skpke), pk := (pksig, pkpke) and pkdet.

(k, ct) ← OMR-KEM.Encap(pk) :
1. (pksig, pkpke) ← pk                       ▷ Parse pk.
2. sig ← OMR.Signal(pksig)                 ▷ Compute the signal.
3. k ←  $\mathcal{K}$                                 ▷ Sample a key k.
4. c ← PKE.Enc(pkpke, k)                   ▷ Encrypt k under pkpke.
5. Return (k, (c, sig))

dgst ← OMR-KEM.Digest(BB, pkdet, P) :
1. dgst ← OMR.Digest(BB, pkdet, P) ▷ Parse each BB element as (c, sig).
2. Return dgst

keys ← OMR-KEM.Decap(dgst, sk) :
1. (skomr, skpke) ← sk                       ▷ Parse sk.
2. keys ← []                                ▷ Initialize empty key list.
3. msg ← Decode(dgst, skomr) ▷ Recover payload list msg from OMR decoding.
4. For each c ∈ msg:
   - keys.append(PKE.Dec(skpke, c)) ▷ Append decryptions.
5. Return keys

```

Figure 4: OMR-KEM construction.

public key encryption scheme PKE and the correctness property of the underlying OMR scheme OMR.

The proof of the above theorem is via a reduction to the correctness property of PKE and OMR. Due to space limitations the proof is deferred for Section C.2.

THEOREM 5.4. *The OMR-KEM scheme of Fig. 4, OMR-KEM, satisfies receiver privacy, assuming the receiver privacy property of the underlying public key encryption scheme PKE and the receiver privacy property of the underlying OMR scheme OMR.*

Due to space limitations the proof of the above theorem is deferred for Section C.3.

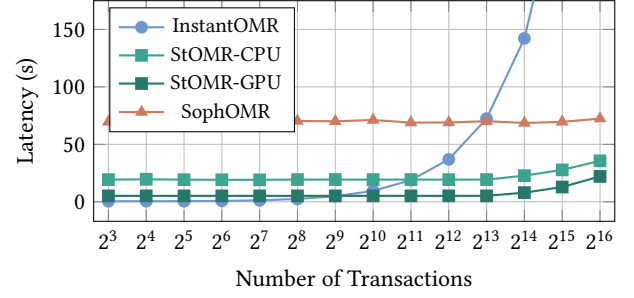
THEOREM 5.5. *The OMR-KEM scheme of Fig. 4, OMR-KEM, is an IND-CPA secure KEM, assuming the IND-CPA security of PKE.*

Due to space limitations the proof of the above theorem is deferred for Section C.4.

6 Evaluation

6.1 Experimental Setup

We implement the FHE-based S-OMR protocol for both CPU and GPU, available at <https://github.com/yiannisdt/StOMR>. StOMR-CPU is implemented using the OpenFHE libraries as a BFV backend while StOMR-GPU uses a Microsoft SEAL-based interface and a custom set of BFV CUDA kernels as the backend. All experiments were run on a server with a 64-core AMD EPYC 7702P, 256 GB of RAM, and an NVIDIA RTX 6000 Ada GPU with 48 GB memory. Where

**Figure 5: Latency comparisons with related works for an increasing number of transactions. Payload x size = 4896 bits and pertinent messages $P = 4$.**

possible, we exploit the concurrency in the protocol to parallelize across multiple threads. We parameterize BFV for StOMR with a polynomial degree of 2^{16} and a plaintext modulus of 20 bits for a security level of at least 128 bits, yielding a multiplicative depth of 22. For SophOMR, to accommodate the range check, we utilize the same polynomial degree, but with six more primes in the ciphertext modulus. We note that this larger parameter set corresponds to approximately 128 bits of security. We fix the size of PRF outputs to be 32 bits for all experiments (each of which generated a tag occupying 2 ciphertext slots). Lastly, in accordance with prior work [41, 42], we fix the payload size to be 612 bytes (4896 bits), which is the typical size of Zcash transactions [36].

Dataset Sizes. To choose realistic bulletin board sizes for evaluation, we rely on public statistics about shielded transactions in Zcash. ZecHub⁴ maintains updated statistics on shielded activity derived from the Zcash blockchain⁵. Using these data, we observe that between November 2024 and November 2025, there were 198,493 Orchard shielded transactions and 263,376 total shielded transactions (Sapling and Orchard combined). Orchard represents the most recent and actively used shielded protocol in Zcash.⁶

Dividing the total number of Orchard transactions by roughly one year of activity yields around 16,000 shielded transactions per month. 2^{14} transactions correspond to roughly about one month of accumulated Orchard activity, making this range representative for typical wallet synchronization. Even in the rare case where more transactions must be processed (e.g., after prolonged inactivity), the bulletin board can be handled by running multiple protocol instances on disjoint subsets of the transactions.

6.2 Comparisons with Related Work

Detector Cost. Fig. 5 depicts the server latency of StOMR and state-of-the-art OMR protocols in the form of SophOMR [41] and InstantOMR [42]. We observe that StOMR-CPU exhibits a lower latency of up to approximately $3.6\times$ relative to SophOMR and has a faster evaluation time across the entire range of transactions for settings with a small number of pertinent messages. StOMR-GPU shows matching scaling, but decreases latency significantly – $1.6\text{--}3.7\times$ faster than StOMR-CPU and $3.3\text{--}13.6\times$ faster relative to

⁴<https://zechub.wiki>

⁵<https://raw.githubusercontent.com/ZecHub/zechub-wiki/refs/heads/main/public/data/zcash/shieldedStatsJSON>

⁶<https://zcash.github.io/orchard>

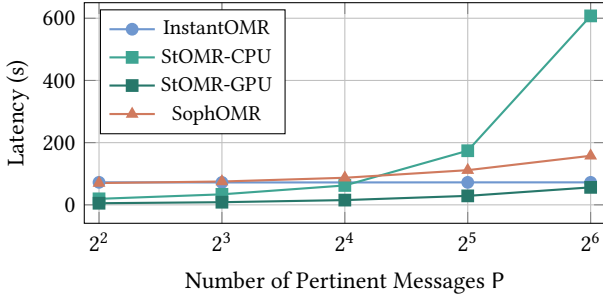


Figure 6: Latency comparisons with related works for an increasing number of pertinent messages P . Transactions = 2^{13} and payload $\times$ size = 4896 bits. StOMR-GPU executes $B = P/4$ parallel batches of queries.

SophOMR. On the other hand, InstantOMR is optimized specifically for settings where the number of transactions on the bulletin board is small; as expected, it exhibits a lower latency than StOMR for these cases. StOMR-CPU and StOMR-GPU outperform InstantOMR in this setting when the size of the bulletin board exceeds about 2048 and 1024 transactions, respectively, and exhibit a speedup of over 15 $\times$ and 25 $\times$, respectively, for 2^{16} transactions.

Additionally, we evaluate each protocol’s performance given an increasing number of pertinent messages (which also corresponds to the number of filters in StOMR) per user in Fig. 6 for a fixed number of transactions and payload size. We find that StOMR excels for settings with a limited number of pertinent messages, as this has a large bearing on the number of slots, and therefore ciphertexts, required to evaluate the protocol. In particular, the computational complexity of the compression step of the protocol scales with this parameter. SophOMR-CPU’s compression step also scales with the number of pertinent messages P , but at a slower rate since fewer ciphertext slots are required to hold both the query and the intermediate data. StOMR-GPU instead runs $B = P/4$ parallel queries with $P = 4$ pertinent messages each, so per-query compression dimensions stay fixed; total work grows only linearly in the number of parallel queries B rather than quadratically in P , and we leverage the GPU to execute the B queries concurrently. As a result, even up to $P = 2^6$, StOMR-GPU’s outperforms each of our baselines. Lastly, InstantOMR scales primarily with the size of the bulletin board, and the number of pertinent messages has a negligible bearing on the overall server latency.

Finally, we evaluate the latency-communication tradeoff for GPU-based parallel queries in Fig. 7. Larger sets of pertinent messages P concentrate work into a single query, reducing the number of query ciphertexts uploaded by the client but driving server latency up significantly; smaller P paired with more parallel batches $B = 64/P$ flattens latency at the cost of an extra ~ 30 MiB query ciphertext per batch. The resulting Pareto frontier has $(P, B) = (16, 4)$ minimizing total bytes of communication for query and response (180 MiB) and $(4, 16)$ minimizing overall server latency (55 s). At the extreme, both large-batch points $(2, 32)$ and $(1, 64)$ suffer from both higher communication and server latency.

Sender Cost. In Table 1 we report the cost of publishing protocol digests on chain under these models. StOMR signal sizes are approximately 550 $\times$, 650 $\times$, and 178 $\times$ smaller than PerfOMR, SophOMR, and

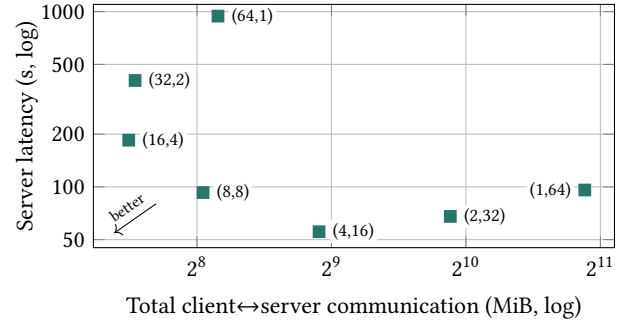


Figure 7: Server-side latency vs. total client-server communication for the (P, B) tradeoff at $P \cdot B = 64$ pertinent messages, with ring-switching enabled. $T = 2^{13}$, payload = 4896 bits; each point is labelled (P, B) .

Table 1: Signal publication cost. Ethereum assumes a 0.50 gwei gas price and 16 gas/byte calldata. Zcash assumes 0.00001 ZEC per kB relay fee.

Protocol	Signal	Gas	ETH Cost	ZEC Cost
PerfOMR	2.2 KB	56,200	2.8100e-5	2.20e-5
SophOMR	2.6 KB	62,600	3.1300e-5	2.60e-5
InstantOMR	714 B	32,424	1.6212e-5	6.97e-6
StOMR	8 B	21,128	1.0564e-5	7.81e-8
StOMR	4 B	21,064	1.0532e-5	3.90e-8

InstantOMR, respectively. To estimate on-chain publication costs, we model signal submission as calldata in a standard Ethereum transaction with calldata costing 16 gas per nonzero byte and 4 gas per zero byte, in addition to the 21,000 gas base transaction fee [1]. For Zcash, we use the standard relay fee model of approximately 0.00001 ZEC per kilobyte, yielding a linear cost in the signal size [62]. Using a conservative gas price of 0.50 gwei, publishing a 4-byte StOMR signal costs approximately 1.0532e-5 ETH, compared to 2.8524e-5 ETH for the 2.2 KB PerfOMR signal, 3.1796e-5 ETH for the 2.6 KB SophOMR signal, and 1.6212e-5 ETH for the 714 B InstantOMR signal. At a representative market price of \$2,050 per ETH and \$222 per ZEC,⁷ the Ethereum calldata costs in Table 1 correspond to \$0.06 for PerfOMR, \$0.07 for SophOMR, \$0.04 for InstantOMR, and \$0.02 for StOMR due to the fixed Ethereum transaction cost. Under the Zcash relay-fee model, where fees scale linearly with signal size, the savings are much more pronounced, with StOMR (\$0.000009) reducing publication costs by approximately 564 $\times$, 667 $\times$, and 179 $\times$ compared to PerfOMR (\$0.005), SophOMR (\$0.006), and InstantOMR (\$0.002), respectively.

References

- [1] Martin Holst Swende Adler et al. 2019. EIP-2028: Transaction Data Gas Cost Reduction. <https://eips.ethereum.org/EIPS/eip-2028>. Ethereum Improvement Proposal.
- [2] Carlos Aguilar-Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. 2016. XPIR: Private Information Retrieval for Everyone. *PoPETs* 2016, 2 (April 2016), 155–174. doi:10.1515/popets-2016-0010

⁷https://www.coingecko.com/en/coins/ethereum/historical_data?start=2026-02-01&end=2026-03-01 and https://www.coingecko.com/en/coins/zcash/historical_data?start=2026-02-01&end=2026-03-01

- [3] Asra Ali, Tancrede Lepoint, Sarvar Patel, Mariana Raykova, Philipp Schoppmann, Karn Seth, and Kevin Yeo. 2021. Communication-Computation Trade-offs in PIR. In *USENIX Security 2021*, Michael Bailey and Rachel Greenstadt (Eds.). USENIX Association, 1811–1828. <https://www.usenix.org/conference/usenixsecurity21/presentation/ali>
- [4] Joël Alwen, Sandro Coretti, Yevgeniy Dodis, and Yiannis Tselekounis. 2020. Security Analysis and Improvements for the IETF MLS Standard for Group Messaging. In *CRYPTO 2020, Part I (LNCS, Vol. 12170)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 248–277. doi:10.1007/978-3-030-56784-2_9
- [5] Joël Alwen, Sandro Coretti, Yevgeniy Dodis, and Yiannis Tselekounis. 2021. Modular Design of Secure Group Messaging Protocols and the Security of MLS. In *ACM CCS 2021*, Giovanni Vigna and Elaine Shi (Eds.). ACM Press, Virtual Event, Republic of Korea, 1463–1483. doi:10.1145/3460120.3484820
- [6] Joël Alwen, Xiaohui Ding, Sanjam Garg, and Yiannis Tselekounis. 2025. Policy Compliant Secure Messaging. In *ASIACRYPT 2025, Part II (LNCS, Vol. 16246)*, Goichiro Hanaoka and Bo-Yin Yang (Eds.). Springer, Singapore, Singapore, Melbourne, VIC, Australia, 34–67. doi:10.1007/978-981-95-5096-8_2
- [7] Joël Alwen, Marta Mularczyk, and Yiannis Tselekounis. 2023. Fork-Resilient Continuous Group Key Agreement. In *CRYPTO 2023, Part IV (LNCS, Vol. 14084)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 396–429. doi:10.1007/978-3-031-38551-3_13
- [8] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. 2018. PIR with Compressed Queries and Amortized Query Processing. In *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 962–979. doi:10.1109/SP.2018.00062
- [9] Sebastian Angel and Srinath Setty. 2016. Unobservable communication over fully untrusted infrastructure. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. 551–569.
- [10] Aztec Protocol Team. 2026. Diversified and Stealth Keys: Example Usage. <https://rknhr-uec.github.io/aztec-protocol-spec/protocol-specs/addresses-and-keys/example-usage/diversified-and-stealth-keys>. Aztec Protocol Specification. Accessed: 2026-03-06.
- [11] Foteini Baldimtsi, Kostas Kryptos Chalkias, Varun Madathil, and Arnab Roy. 2024. SoK: Privacy-Preserving Transactions in Blockchains. *Cryptology ePrint Archive*, Report 2024/1959. <https://eprint.iacr.org/2024/1959>
- [12] Gabrielle Beck, Julia Len, Ian Miers, and Matthew Green. 2021. Fuzzy Message Detection. In *ACM CCS 2021*, Giovanni Vigna and Elaine Shi (Eds.). ACM Press, Virtual Event, Republic of Korea, 1507–1528. doi:10.1145/3460120.3484545
- [13] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. 2014. Zerocash: Decentralized Anonymous Payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, Berkeley, CA, USA, 459–474. doi:10.1109/SP.2014.36
- [14] Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. 2024. Batch PIR and Labeled PSI with Oblivious Ciphertext Compression. In *USENIX Security 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association, Philadelphia, PA, USA. <https://www.usenix.org/conference/usenixsecurity24/presentation/bienstock>
- [15] Andrea Bittau, Úlfar Erlingsson, Petros Maniatis, Ilya Mironov, Ananth Raghunathan, David Lie, Mitch Rudominer, Ushasree Kode, Julien Tinnes, and Bernhard Seefeld. 2017. Prochlo: Strong Privacy for Analytics in the Crowd. In *Proceedings of the 26th Symposium on Operating Systems Principles (Shanghai, China) (SOSP '17)*. Association for Computing Machinery, New York, NY, USA, 441–459. doi:10.1145/3132747.3132769
- [16] Zvika Brakerski. 2012. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *CRYPTO 2012 (LNCS, Vol. 7417)*, Reihaneh Safavi-Naini and Ran Canetti (Eds.). Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA, 868–886. doi:10.1007/978-3-642-32009-5_50
- [17] Vitalik Buterin et al. 2013. Ethereum white paper. *GitHub repository* 1, 22-23 (2013), 5–7.
- [18] Sofia Celi and Alex Davidson. 2024. Call Me By My Name: Simple, Practical Private Information Retrieval for Keyword Queries. In *ACM CCS 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM Press, Salt Lake City, UT, USA, 4107–4121. doi:10.1145/3658644.3670271
- [19] Jung Hee Cheon, Keewoo Lee, Jai Hyun Park, and Yongdong Yeo. 2024. SIMD-Aware Homomorphic Compression and Application to Private Database Query. arXiv:2408.17063 [cs.CR] <https://arxiv.org/abs/2408.17063> <https://arxiv.org/abs/2408.17063>
- [20] Seung Geol Choi, Dana Dachman-Soled, S. Dov Gordon, Linsheng Liu, and Arkady Yerukhimovich. 2021. Compressed Oblivious Encoding for Homomorphically Encrypted Search. In *ACM CCS 2021*, Giovanni Vigna and Elaine Shi (Eds.). ACM Press, Virtual Event, Republic of Korea, 2277–2291. doi:10.1145/3460120.3484792
- [21] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. 1995. Private Information Retrieval. In *36th FOCS*. IEEE Computer Society Press, Milwaukee, Wisconsin, 41–50. doi:10.1109/SFCS.1995.492461
- [22] Haotian Chu, Xiao Wang, and Yanxue Jia. 2025. Private Signaling Secure Against Actively Corrupted Servers. *Cryptology ePrint Archive*, Report 2025/1056. <https://eprint.iacr.org/2025/1056>
- [23] Henry Corrigan-Gibbs and Dan Boneh. 2017. Prio: Private, Robust, and Scalable Computation of Aggregate Statistics. In *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation (Boston, MA, USA) (NSDI'17)*. USENIX Association, USA, 259–282.
- [24] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. 2015. Riposte: An Anonymous Messaging System Handling Millions of Users. In *2015 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Jose, CA, USA, 321–338. doi:10.1109/SP.2015.27
- [25] Henry Corrigan-Gibbs, Alexandra Henzinger, and Dmitry Kogan. 2022. Single-Server Private Information Retrieval with Sublinear Amortized Time. In *EUROCRYPT 2022, Part II (LNCS, Vol. 13276)*, Orr Dunkelman and Stefan Dziembowski (Eds.). Springer, Cham, Switzerland, Trondheim, Norway, 3–33. doi:10.1007/978-3-031-07085-3_1
- [26] Leo de Castro and Keewoo Lee. 2024. VeriSimplePIR: Verifiability in SimplePIR at No Online Cost for Honest Servers. In *USENIX Security 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association, Philadelphia, PA, USA. <https://www.usenix.org/conference/usenixsecurity24/presentation/de-castro>
- [27] Yevgeniy Dodis, Bernardo Magri, Noah Stephens-Davidowitz, and Yiannis Tselekounis. 2025. Guarding the Signal: Secure Messaging with Reverse Firewalls. In *CRYPTO 2025, Part VIII (LNCS, Vol. 16007)*, Yael Tauman Kalai and Seny F. Kamara (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 39–74. doi:10.1007/978-3-032-01913-4_2
- [28] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Report 2012/144. <https://eprint.iacr.org/2012/144>
- [29] Nils Fleischhacker, Kasper Green Larsen, and Mark Simkin. 2023. How to Compress Encrypted Data. In *EUROCRYPT 2023, Part I (LNCS, Vol. 14004)*, Carmi Hazay and Martijn Stam (Eds.). Springer, Cham, Switzerland, Lyon, France, 551–577. doi:10.1007/978-3-031-30545-0_19
- [30] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *41st ACM STOC*, Michael Mitzenmacher (Ed.). ACM Press, Bethesda, MD, USA, 169–178. doi:10.1145/1536414.1536440
- [31] Christopher Goes, Yulia Khalniyazova, Enrique Larraia, and Xuyang Song. 2025. Multi-server Fuzzy Message Detection. *Cryptology ePrint Archive*, Report 2025/2072. <https://eprint.iacr.org/2025/2072>
- [32] Charles Gouert, Dimitris Mouris, and Nektarios Georgios Tsoutsos. 2023. SoK: New Insights into Fully Homomorphic Encryption Libraries via Standardized Benchmarks. *PopETs 2023*, 3 (July 2023), 154–172. doi:10.56553/popets-2023-0075
- [33] Meng Hao, Weiran Liu, Liqiang Peng, Cong Zhang, Pengfei Wu, Lei Zhang, Hongwei Li, and Robert H. Deng. 2025. Practical Keyword Private Information Retrieval from Key-to-Index Mappings. In *USENIX Security 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, Seattle, WA, USA, 3397–3416. <https://www.usenix.org/conference/usenixsecurity25/presentation/haohao>
- [34] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. 2023. One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval. In *USENIX Security 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, Anaheim, CA, USA, 3889–3905. <https://www.usenix.org/conference/usenixsecurity23/presentation/henzinger>
- [35] Alexander Hoover, Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. 2025. Plinko: Single-Server PIR with Efficient Updates via Invertible PRFs. In *EUROCRYPT 2025, Part VI (LNCS, Vol. 15606)*, Serge Fehr and Pierre-Alain Fouque (Eds.). Springer, Cham, Switzerland, Madrid, Spain, 3–33. doi:10.1007/978-3-031-91095-1_1
- [36] Daira Hopwood, Sean Bowe, Taylor Hornby, Nathan Wilcox, et al. 2016. Zcash protocol specification. *GitHub: San Francisco, CA, USA* 4, 220 (2016), 32.
- [37] Ilia Iliashenko and Vincent Zucca. 2021. Faster homomorphic comparison operations for BGV and BFV. *PopETs 2021*, 3 (July 2021), 246–264. doi:10.2478/popets-2021-0046
- [38] Sashidhar Jakkamsetti, Zeyu Liu, and Varun Madathil. 2025. Scalable Private Signaling. In *2025 IEEE 38th Computer Security Foundations Symposium (CSF)*. 143–158. doi:10.1109/CSF64896.2025.00035
- [39] Yanxue Jia, Varun Madathil, and Aniket Kate. 2024. HomeRun: High-efficiency Oblivious Message Retrieval, Unrestricted. In *ACM CCS 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM Press, Salt Lake City, UT, USA, 2012–2026. doi:10.1145/3658644.3670381
- [40] Allen Klinger. 1967. The vandermonde matrix. *The American Mathematical Monthly* 74, 5 (1967), 571–574.
- [41] Keewoo Lee and Yongdong Yeo. 2026. SophOMR: Improved Oblivious Message Retrieval from SIMD-Aware Homomorphic Compression. In *USENIX Security 2026*. USENIX Association.
- [42] Haofei Liang, Zeyu Liu, Eran Tromer, Xiang Xie, and Yu Yu. 2026. InstantOMR: Oblivious Message Retrieval with Low Latency and Optimal Parallelizability. In *USENIX Security 2026*. USENIX Association.
- [43] Yini Lin and Haibo Tian. 2025. Efficient Updatable Private Information Retrieval From Simulatable Homomorphic Ciphertexts. In *ASIACCS 25*. ACM Press, Hanoi, Vietnam, 45–57. doi:10.1145/3708821.3733871

- [44] Zeyu Liu, Katerina Sotiraki, Eran Tromer, and Yunhao Wang. 2025. Snake-Eye Resistant PKE from LWE for Oblivious Message Retrieval and Robust Encryption. In *EUROCRYPT 2025, Part III (LNCS, Vol. 15603)*, Serge Fehr and Pierre-Alain Fouque (Eds.). Springer, Cham, Switzerland, Madrid, Spain, 126–156. doi:10.1007/978-3-031-91131-6_5
- [45] Zeyu Liu and Eran Tromer. 2022. Oblivious Message Retrieval. In *CRYPTO 2022, Part I (LNCS, Vol. 13507)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 753–783. doi:10.1007/978-3-031-15802-5_26
- [46] Zeyu Liu, Eran Tromer, and Yunhao Wang. 2024. Group Oblivious Message Retrieval. In *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 4367–4385. doi:10.1109/SP54263.2024.00115
- [47] Zeyu Liu, Eran Tromer, and Yunhao Wang. 2024. PerfOMR: Oblivious Message Retrieval with Reduced Communication and Computation. In *USENIX Security 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association, Philadelphia, PA, USA. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-zeyu>
- [48] Yiping Ma, Ke Zhong, Tal Rabin, and Sebastian Angel. 2022. Incremental Offline/Online PIR. In *USENIX Security 2022*, Kevin R. B. Butler and Kurt Thomas (Eds.). USENIX Association, Boston, MA, USA, 1741–1758. <https://www.usenix.org/conference/usenixsecurity22/presentation/ma>
- [49] Varun Madathil, Alessandra Scafuro, István András Seres, Omer Shlomovits, and Denis Varlakov. 2022. Private Signaling. In *USENIX Security 2022*, Kevin R. B. Butler and Kurt Thomas (Eds.). USENIX Association, Boston, MA, USA, 3309–3326. <https://www.usenix.org/conference/usenixsecurity22/presentation/madathil>
- [50] Rasoul Akhavan Mahdavi and Florian Kerschbaum. 2022. Constant-weight PIR: Single-round Keyword PIR via Constant-weight Equality Operators. In *USENIX Security 2022*, Kevin R. B. Butler and Kurt Thomas (Eds.). USENIX Association, Boston, MA, USA, 1723–1740. <https://www.usenix.org/conference/usenixsecurity22/presentation/mahdavi>
- [51] Dimitris Mouris, Christopher Patton, Hannah Davis, Pratik Sarkar, and Nektarios Georgios Tsoutsos. 2025. Mastic: Private Weighted Heavy-Hitters and Attribute-Based Metrics. *POPETs 2025*, 1 (Jan. 2025), 290–319. doi:10.56553/popets-2025-0017
- [52] Satoshi Nakamoto. 2008. Bitcoin: A peer-to-peer electronic cash system. (2008).
- [53] Shen Noether. 2015. Ring Signature Confidential Transactions for Monero. *Cryptology ePrint Archive*, Report 2015/1098. <https://eprint.iacr.org/2015/1098>
- [54] Sarvar Patel, Joon Young Seo, and Kevin Yeo. 2023. Don't be Dense: Efficient Keyword PIR for Sparse Databases. In *USENIX Security 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, Anaheim, CA, USA, 3853–3870. <https://www.usenix.org/conference/usenixsecurity23/presentation/patel>
- [55] Sihang Pu, Sri Aravinda Krishnan Thyagarajan, Nico Döttling, and Lucjan Hanzlik. 2023. Post Quantum Fuzzy Stealth Signatures and Applications. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, Copenhagen, Denmark, 371–385. doi:10.1145/3576915.3623148
- [56] Signal. 2018. Technology Preview: Sealed Sender for Signal. <https://signal.org/blog/sealed-sender/>. Signal Blog.
- [57] Nigel P. Smart and Frederik Vercauteren. 2014. Fully homomorphic SIMD operations. *DCC 71*, 1 (2014), 57–81. doi:10.1007/s10623-012-9720-4
- [58] Jelle van den Hooff, David Lazar, Matei Zaharia, and Nickolai Zeldovich. 2015. Vuvuzela: scalable private messaging resistant to traffic analysis. In *Proceedings of the 25th Symposium on Operating Systems Principles (Monterey, California) (SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 137–152. doi:10.1145/2815400.2815417
- [59] Nicolas van Saberhagen. 2013. *CryptoNote v2.0*. <https://cryptonote.org/whitepaper.pdf>. Accessed: 2026-03-06.
- [60] Alexander Viand, Patrick Jattke, and Anwar Hithnawi. 2021. SoK: Fully Homomorphic Encryption Compilers. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 1092–1108. doi:10.1109/SP40001.2021.00068
- [61] Anton Wahrstätter, Matthew Solomon, Ben DiFrancesco, Vitalik Buterin, and Davor Svetinovic. 2024. BaseSAP: Modular Stealth Address Protocol for Programmable Blockchains. *Trans. Info. For. Sec.*, 19 (Jan. 2024), 3539–3553. doi:10.1109/TIFS.2024.3364081
- [62] Zcash Foundation. 2022. ZIP 317: Fee Calculation. <https://zips.z.cash/zip-0317>.
- [63] Mingxun Zhou, Wei-Kai Lin, Yiannis Tselekounis, and Elaine Shi. 2023. Optimal Single-Server Private Information Retrieval. In *EUROCRYPT 2023, Part I (LNCS, Vol. 14004)*, Carmit Hazay and Martijn Stam (Eds.). Springer, Cham, Switzerland, Lyon, France, 395–425. doi:10.1007/978-3-031-30545-0_14
- [64] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. 2024. Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation. In *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 4296–4314. doi:10.1109/SP54263.2024.00055

A Open Science

We provide both StOMR's CPU and GPU implementations at <https://github.com/yiannisdt/StOMR>, which contains the source code, build configuration, documentation, and evaluation harnesses used to produce paper figures. StOMR evaluates on randomly-generated synthetic bulletin board entries, matching realistic sizes as detailed in the paper's evaluation, thus does not require any additional datasets. No human-subject or otherwise sensitive data is involved.

Required hardware. For the GPU-based experiments, access to an NVIDIA GPU with compute capability ≥ 8.0 (Ampere or newer) and $\geq 40\text{GB}$ device memory is required.

B Acknowledgements

We were assisted by ChatGPT in performing spelling and grammar checks in this manuscript.

C Deferred Proofs

C.1 Proof of Theorem 5.1

For any $T \in \mathbb{N}$, consider any sequence of honestly generated signals

$$(\text{sig}_i, \text{st}_{\text{snd}}^i) \leftarrow \text{Signal}(\text{st}_{\text{snd}}^{i-1}), \quad i \in [T],$$

and let $\text{BB} = ((x_1, \text{sig}_1), \dots, (x_T, \text{sig}_T))$ be a bulletin board containing these pairs (and possibly additional arbitrary entries). Furthermore, let $\overline{\text{msg}}$ be the list of payloads x_i whose signals sig_i were honestly generated from the evolving sender's state st_{snd} , and for starters assume that $P \geq |\overline{\text{msg}}|$.

Towards contradiction, assume the correctness property of the scheme is violated. Then, with non-negligible probability, $\text{msg} \neq \overline{\text{msg}}$, where msg is the output of Decode. Let E be such an event and let E' be the event in which a signal included in BB that has been generated by another (honest) user matches one of the signals in $(\text{sig}_1, \dots, \text{sig}_T)$.

Recall that signals are computed via PRF evaluations over a private key k that is shared between the sender and the receiver, and the detection algorithm checks equality of signals submitted by the receiver via pk_{det} , with those stored in BB . Therefore, conditioned on $\neg E'$ (and assuming $P \geq |\overline{\text{msg}}|$), each queried signal matches at most one entry in BB , therefore the detector evaluates the correct equality check and the probability of E is upper bounded by the probability of breaking correctness of the underlying FHE scheme, since correctness could only be violated if the FHE decryption fails to output the right payloads. Therefore $\Pr[E \wedge \neg E'] \leq \text{negl}(\lambda)$, assuming the correctness property of the underlying FHE scheme.

We now bound $\Pr[E \wedge E']$ by bounding $\Pr[E']$. Let Hyb_0 be the execution defined by the correctness property. We define Hyb_1 to be as Hyb_0 with the only difference being that signals of the target sender/receiver pair (the pair for which we prove correctness) are computed by simulating a uniformly random function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$. Clearly, Hyb_0 is indistinguishable from Hyb_1 , assuming the pseudorandomness of PRF. Furthermore, in Hyb_1 , for any j , $\Pr[f(j) = \text{sig}] = \frac{1}{2^m}$, since f is picked uniformly at random. Let E'' be the event of having a collision in any pair of signals in BB . Then, we have $\Pr[E'] \leq \Pr[E''] \leq \binom{|\text{BB}|}{2} \cdot \frac{1}{2^m} \leq |\text{BB}|^2 \cdot \frac{1}{2^m}$, and the proof is concluded, for $P \geq |\overline{\text{msg}}|$.

For $P < |\widetilde{\text{msg}}|$, observe that during the decode operation the protocol prepares P signals for the next digest operation, and correctness of subsequent digest executions is handled as above. In that case the receiver requests multiple digest operations in order to recover all the incoming payloads.

C.2 Proof of Theorem 5.3

Towards contradiction, assume the correctness of the scheme of Fig. 4 is violated. Then by definition (cf. Fig. 3), with non-negligible probability, $\text{keys} \neq \text{keys}'$, where keys denotes the list of keys generated via an honest execution of the encapsulation operation and keys' denotes the list of keys computed after decapsulation. Then, there exists $i \in [T]$, such that $(\tilde{k}_i, \text{ct}_i)$ was honestly generated, and the i -th entry of keys , k_i , is not equal to $\tilde{k}_i$. By definition ct_i , consists of (c_i, sig_i) , therefore $\tilde{k}_i \neq k_i$, implies either that the i -th entry of msg (the list of decoded ciphertexts) is not equal to the PKE ciphertext c_i , and clearly this would violate the correctness property of OMR since by assumption the payload of the i -th encapsulation operation is c_i , or $\tilde{k}_i \neq \text{PKE.Dec}(\text{sk}_{\text{pke}}, c_i)$, violating the correctness of PKE, since by assumption c_i is encrypting $\tilde{k}_i$. Therefore, the probability of breaking the correctness property of OMR-KEM is upper bounded by the probability of breaking the correctness property of the underlying schemes.

C.3 Proof of Theorem 5.4

The proof follows a sequence of hybrid experiments, described below.

Hybrid Hyb₀. This hybrid matches $\text{Exp}_{\text{RP},0}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A})$, where $\text{Exp}_{\text{RP},b}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A})$ is defined as follows:

$\text{Exp}_{\text{RP},b}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A})$:

- $\text{pp} \leftarrow \text{ParamGen}(1^\lambda)$,
- $(\text{sk}_0, \text{pk}_0, \text{pk}_{\text{det}}^0) \leftarrow \text{KeyGen}(\text{pp})$,
- $(\text{sk}_1, \text{pk}_1, \text{pk}_{\text{det}}^1) \leftarrow \text{KeyGen}(\text{pp})$,
- $(\cdot, \text{ct}_0) \leftarrow \text{Encap}(\text{pk}_0)$; $(\cdot, \text{ct}_1) \leftarrow \text{Encap}(\text{pk}_1)$,
- $\text{aux} \leftarrow (\text{pp}, \text{pk}_0, \text{pk}_{\text{det}}^0, \text{pk}_1, \text{pk}_{\text{det}}^1)$,
- $b' \leftarrow \mathcal{A}(\text{aux}, \text{ct}_b)$,
- Return b' .

Hybrid Hyb₁. By the definition of OMR-KEM $\text{pk}_0 = (\text{pk}_{\text{sig}}^0, \text{pk}_{\text{pke}}^0)$, $\text{pk}_1 = (\text{pk}_{\text{sig}}^1, \text{pk}_{\text{pke}}^1)$. Hyb₁ computes $(\cdot, \text{ct}) \leftarrow \text{Encap}(\text{pk})$, where $\text{pk} = (\text{pk}_{\text{sig}}^0, \text{pk}_{\text{pke}}^0)$, and ct is provided to $\mathcal{A}$ together with $\text{aux} \leftarrow (\text{pp}, \text{pk}_0, \text{pk}_{\text{det}}^0, \text{pk}_1, \text{pk}_{\text{det}}^1)$. The indistinguishability between the two hybrids reduces to the receiver privacy security of PKE.

More formally, we define an adversary $\mathcal{A}'$ that plays against the receiver privacy game w.r.t. PKE. $\mathcal{A}'$ receives $\text{pk}_{\text{pke}}^0, \text{pk}_{\text{pke}}^1$ and computes the following:

- $\text{pp} \leftarrow \text{OMR.ParamGen}(1^\lambda)$
- $(\text{sk}_{\text{omr}}, \text{pk}_{\text{sig}}^0, \text{pk}_{\text{det}}^0) \leftarrow \text{OMR.KeyGen}(\text{pp})$,
- $(\text{sk}_{\text{omr}}, \text{pk}_{\text{sig}}^1, \text{pk}_{\text{det}}^1) \leftarrow \text{OMR.KeyGen}(\text{pp})$,
- Sample $k \leftarrow \mathcal{K}$ and return $(k, \perp)$ to challenger of $\text{Exp}_{\text{RP},b}^{\text{PKE}}$,
- On input $(\perp, c)$ from $\text{Exp}_{\text{RP},b}^{\text{PKE}}$, compute $\text{sig} \leftarrow \text{OMR.Signal}(\text{pk}_{\text{sig}}^0)$,
- Set $\text{ct} \leftarrow (c, \text{sig})$,
- Set $\text{pk}_0 \leftarrow (\text{pk}_{\text{sig}}^0, \text{pk}_{\text{pke}}^0)$, $\text{pk}_1 \leftarrow (\text{pk}_{\text{sig}}^1, \text{pk}_{\text{pke}}^1)$,

- $\text{aux} \leftarrow (\text{pp}, \text{pk}_0, \text{pk}_{\text{det}}^0, \text{pk}_1, \text{pk}_{\text{det}}^1)$,
- Compute $b' \leftarrow \mathcal{A}'(\text{aux}, \text{ct})$ and return b' .

Clearly, if $\mathcal{A}'$ plays against $\text{Exp}_{\text{RP},0}^{\text{PKE}}$, then it perfectly simulates Hyb₀, while if it plays against $\text{Exp}_{\text{RP},1}^{\text{PKE}}$, it simulates perfectly Hyb₁, and the distinguishing advantage of $\mathcal{A}'$ against Hyb₀, Hyb₁, is bounded by the distinguishing advantage of $\mathcal{A}$ against $\text{Exp}_{\text{RP},0}^{\text{PKE}}, \text{Exp}_{\text{RP},1}^{\text{PKE}}$, which is negligible by the receiver privacy security of PKE.

Hybrid Hyb₂. This hybrid matches $\text{Exp}_{\text{RP},1}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A})$. The indistinguishability between the current hybrid and Hyb₁ reduces to the receiver privacy security of OMR. More formally, we define an adversary $\mathcal{A}'$ that plays against the receiver privacy game w.r.t. OMR. $\mathcal{A}'$ receives $\text{aux} := (\text{pp}, \text{pk}_0, \text{pk}_{\text{det}}^0, \text{pk}_1, \text{pk}_{\text{det}}^1)$ and sig from the $\text{Exp}_{\text{RP},b}^{\text{OMR}}$ and computes the following:

- $(\text{sk}_{\text{pke}}, \text{pk}_{\text{pke}}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$
- Set $\text{pk}'_0 \leftarrow (\text{pk}_0, \text{pk}_{\text{pke}})$, $\text{pk}'_1 \leftarrow (\text{pk}_1, \text{pk}_{\text{pke}})$
- $k \leftarrow \mathcal{K}$, $c \leftarrow \text{PKE.Enc}(\text{pk}_{\text{pke}}, k)$
- Set $\text{aux}' \leftarrow (\text{pp}, \text{pk}'_0, \text{pk}_{\text{det}}^0, \text{pk}'_1, \text{pk}_{\text{det}}^1)$, $\text{ct} \leftarrow (c, \text{sig})$.
- Compute $b' \leftarrow \mathcal{A}'(\text{aux}', \text{ct})$ and return b' .

Clearly, if $\mathcal{A}'$ plays against $\text{Exp}_{\text{RP},0}^{\text{OMR}}$ then it perfectly simulates Hyb₁ for $\mathcal{A}$, otherwise it perfectly simulates Hyb₂. Therefore, the distinguishing advantage of $\mathcal{A}$ is bounded by the distinguishing advantage of $\mathcal{A}'$ against the receiver privacy security of OMR.

C.4 Proof of Theorem 5.5

Let $\mathcal{A}$ be any adversary against the IND-CPA KEM security of OMR. The proof is via a sequence of hybrids.

Hybrid Hyb₀. This hybrid matches $\text{Exp}_{\text{IND-CPA},0}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A})$.

Hybrid Hyb₁. In Hyb₁, the Encap operation, instead of encrypting k_0 , encrypts the all zeros key. The indistinguishability between Hyb₀ and Hyb₁ reduces to the IND-CPA security of PKE.

More formally, we define an adversary $\mathcal{A}'$ that plays the IND-CPA security game w.r.t. PKE while simulating Hyb₀ (or Hyb₁) to $\mathcal{A}$. $\mathcal{A}'$ receives the public key of PKE, samples keys for the underlying OMR, and for the encapsulation operation, samples $k_0 \leftarrow \mathcal{K}$, sets $k'_0 := 0^\lambda$, and sends k_0, k'_0 to the challenger of the IND-CPA security game w.r.t. PKE. After receiving c from the challenger, it simulates sig according to the protocol specification, and executes Hyb₀ for the rest of the execution, using $k_0, (c, \text{sig})$, and the OMR-related keys sampled earlier. Clearly, if the IND-CPA security game encrypts k_0 , $\mathcal{A}'$ simulates Hyb₀ for $\mathcal{A}$, otherwise simulates Hyb₁, therefore the distinguishing advantage of $\mathcal{A}$ is bounded by the advantage of $\mathcal{A}'$ in breaking the IND-CPA security of PKE.

Hybrid Hyb₂. Hyb₂ provides k_1 as adversary input. The remaining execution follows Hyb₁. The statistical distance between the two hybrids is negligible, since in both hybrids the adversary receives a uniformly random key, and the ciphertext encrypting 0.

Hybrid Hyb₃. The final hybrid matches $\text{Exp}_{\text{IND-CPA},1}^{\text{OMR-KEM}}(1^\lambda, \mathcal{A})$, i.e., the ciphertext output by the encapsulation operation now encrypts k_0 (instead of 0). The indistinguishability between the current hybrid and the previous one follows from the IND-CPA security of PKE and the proof is along the lines of the one used to prove indistinguishability between Hyb₀ and Hyb₁.